TRƯỜNG ĐẠI HỌC SƯ PHẠM THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO NHÓM

Môn học: 2611COMP101904 – Lập trình trên Windows

Đề tài 14

QUẢN LÝ NHÀ THUỐC (PHARMACARE)

Giảng viên hướng dẫn: Lê Thanh Thoại
Nhóm thực hiện: Nhóm 06
Mã nguồn: https://github.com/cubo40925-prog/Nhom06_QuanLyNhaThuoc

Danh sách thành viên

Họ và tên	MSSV	Vai trò
Nguyễn Đan Trường	51.01.104.110	Thành viên
Phùng Đăng Khoa	51.01.104.046	Thành viên
Nguyễn Trọng Đức Hậu	51.01.104.025	Thành viên
Lê Quốc Tuấn	51.01.104.115	Thành viên

TP. Hồ Chí Minh, Học kỳ 1, Năm học 2026 – 2027

Lời cảm ơn

Nhóm xin chân thành cảm ơn Lê Thanh Thoại đã tận tình hướng dẫn, góp ý về yêu cầu, thiết kế và cách tổ chức mã nguồn trong suốt quá trình thực hiện đề tài. Nhóm cũng cảm ơn Khoa Công nghệ Thông tin, Trường Đại học Sư phạm Thành phố Hồ Chí Minh đã tạo điều kiện về kiến thức nền tảng và môi trường thực hành. Do thời gian và kinh nghiệm còn hạn chế, báo cáo và sản phẩm khó tránh khỏi thiếu sót; nhóm rất mong nhận được ý kiến đóng góp để hoàn thiện hơn.

Nhận xét của giảng viên

Mục lục

Lời cảm ơn i

Nhận xét của giảng viên ii

Danh mục từ viết tắt ix

1 Giới thiệu bài toán và nhu cầu thực tế 1

1.1 Đặt vấn đề . . . 1
1.2 Mục tiêu đề tài . . . 1
1.2.1 Mục tiêu chung . . . 1
1.2.2 Mục tiêu cụ thể . . . 2
1.3 Phạm vi chức năng và đối tượng sử dụng . . . 2
1.4 Phương pháp thực hiện . . . 2
1.5 Công cụ và môi trường . . . 2
1.6 Bố cục báo cáo . . . 2

2 Cơ sở lý thuyết 4

2.1 .NET và ngôn ngữ C# . . . 4
2.2 Lập trình hướng đối tượng . . . 4
2.3 Windows Forms . . . 5
2.4 SQL Server và T-SQL . . . 5
2.5 ADO.NET và phòng chống SQL Injection . . . 6
2.6 Mô hình MVP và kiến trúc nhiều lớp . . . 6
2.7 Giao dịch cơ sở dữ liệu và tính ACID . . . 7
2.8 Phương pháp quản lý kho FEFO . . . 7
2.9 Bảo mật mật khẩu . . . 7

3 Phân tích người dùng, vai trò và phạm vi chức năng 9

3.1 Khảo sát nghiệp vụ . . . 9
3.2 Tác nhân và phân quyền . . . 9
3.3 Yêu cầu chức năng . . . 10
3.4 Yêu cầu phi chức năng . . . 10

3.5 Quy tắc nghiệp vụ . . . 11
3.6 Mô hình ca sử dụng . . . 11

4 Thiết kế cơ sở dữ liệu 13
4.1 Mô hình thực thể quan hệ . . . 13
4.2 Mô tả chi tiết các bảng . . . 13
4.3 Ràng buộc toàn vẹn và chỉ mục . . . 16
4.4 Thủ tục lưu sẵn . . . 16
4.5 Dữ liệu mẫu . . . 18

5 Kiến trúc chương trình 19
5.1 Tổng quan kiến trúc . . . 19
5.2 Tổ chức thư mục mã nguồn . . . 19
5.3 Trách nhiệm từng lớp . . . 20
5.4 Thiết kế lớp . . . 20
5.4.1 Hệ thống phân cấp người dùng . . . 20
5.4.2 Hệ thống ngoại lệ nghiệp vụ . . . 22
5.5 Luồng xử lý chính . . . 23
5.5.1 Luồng đăng nhập . . . 23
5.5.2 Luồng bán hàng theo FEFO . . . 23
5.5.3 Luồng nhập kho . . . 23
5.6 Phân quyền hai tầng . . . 25
5.7 Cơ chế sự kiện làm mới dữ liệu . . . 25

6 Mô tả các chức năng chính 26
6.1 Khởi tạo và tự cấu hình cơ sở dữ liệu . . . 26
6.2 Bảo mật mật khẩu . . . 27
6.3 Thuật toán bán theo FEFO . . . 28
6.4 Giao dịch bán hàng và trừ kho . . . 29
6.4.1 Xử lý kết quả ở Presenter . . . 30
6.5 Nhập kho theo lô . . . 31
6.6 Cảnh báo hạn dùng . . . 32
6.7 In hóa đơn nhiều trang . . . 32
6.8 Quản lý danh mục thuốc và ảnh . . . 33
6.9 Kiểm tra dữ liệu nhập . . . 33

7 Thiết kế giao diện 34
7.1 Màn hình đăng nhập . . . 34
7.2 Trang tổng quan . . . 35
7.3 Danh mục thuốc . . . 35

7.4 Thêm và sửa thuốc . . . 35
7.5 Nhóm thuốc và nhà cung cấp . . . 35
7.6 Nhập kho theo lô . . . 39
7.7 Lịch sử phiếu nhập . . . 39
7.8 Bán hàng . . . 39
7.9 Xem trước và in hóa đơn . . . 39
7.10 Tra cứu hóa đơn . . . 42
7.11 Cảnh báo hạn dùng . . . 42
7.12 Báo cáo doanh thu . . . 43

8 Kiểm thử và kết quả **44**
8.1 Môi trường và phương pháp . . . 44
8.2 Bảng ca kiểm thử . . . 44
8.3 Kết quả thực hiện . . . 46
8.4 Các lỗi phát hiện khi rà soát và cách khắc phục . . . 46
8.5 Đánh giá . . . 48

9 Kết luận, hạn chế và hướng phát triển **49**
9.1 Kết quả đạt được . . . 49
9.2 Hạn chế . . . 49
9.3 Hướng phát triển . . . 50

A Phụ lục cài đặt, tài khoản demo và script cơ sở dữ liệu **51**
A.1 Yêu cầu . . . 51
A.2 Các bước . . . 51
A.3 Chạy script thủ công . . . 51
A.4 Sự cố thường gặp . . . 52

B Đóng góp của các thành viên **53**

C Mã nguồn tiêu biểu **55**
C.1 Tầng Service: `BanHangService.cs` . . . 55
C.2 Tầng Repository: `BanHangRepository.cs` . . . 56
C.3 Presenter: `BanHangPresenter.cs` . . . 58
C.4 In hóa đơn: `InvoiceRenderer.cs` . . . 59
C.5 Khởi tạo CSDL: `DbSetup.cs` . . . 61
C.6 Script tạo cơ sở dữ liệu . . . 63
C.7 Thủ tục lưu sẵn . . . 65

Tài liệu tham khảo **68**

Danh sách hình vẽ

3.1 Sơ đồ ca sử dụng (Use Case) của hệ thống PharmaCare 12

4.1 Sơ đồ thực thể quan hệ (ERD) của PharmaCareDB 14

5.1 Kiến trúc MVP kết hợp nhiều lớp của PharmaCare 19
5.2 Sơ đồ lớp (Class Diagram) – Models, Services và Repositories chính 22
5.3 Sơ đồ tuần tự: đăng nhập 23
5.4 Sơ đồ tuần tự: bán hàng và thanh toán 24
5.5 Sơ đồ tuần tự: nhập kho theo lô 24

7.1 Màn hình đăng nhập 34
7.2 Trang tổng quan 35
7.3 Danh mục thuốc (tài khoản Quản lý kho) 36
7.4 Danh mục thuốc với tài khoản Dược sĩ: các nút chỉnh sửa bị khóa 36
7.5 Biểu mẫu thêm thuốc 37
7.6 Báo lỗi nhập liệu ngay tại ô (`ErrorProvider`) 37
7.7 Trang Nhóm thuốc 38
7.8 Trang Nhà cung cấp 38
7.9 Trang Lô nhập kho 39
7.10 Trang Phiếu nhập (lịch sử nhập kho) 40
7.11 Trang Bán hàng với giỏ hàng phân bổ nhiều lô 40
7.12 Báo lỗi khi bán vượt tồn: biểu tượng lỗi đỏ cạnh ô Số lượng, nội dung (“Chỉ còn bán được tối đa 8”) hiện khi rê chuột 41
7.13 Xem trước hóa đơn (Print Preview): hóa đơn nhiều dòng tự sang trang thứ hai, tổng cộng ở trang cuối 41
7.14 Trang Hóa đơn 42
7.15 Trang Cảnh báo hạn dùng với các lô tô màu 42
7.16 Báo cáo doanh thu theo ngày 43

8.1 Minh chứng một số ca kiểm thử: TC03 (sai mật khẩu), TC04 (thử SQL injection), TC06 (trùng mã), TC10 (không cho xóa thuốc đã có lô) và TC16 (bán vượt tồn) 48

B.1 Thống kê commit của các thành viên trên GitHub 54

Danh sách bảng

1.1 Công cụ và môi trường phát triển 3

3.1 Ma trận phân quyền theo vai trò 9
3.2 Danh sách yêu cầu chức năng 10

4.1 Bảng Users – tài khoản đăng nhập 13
4.2 Bảng Categories – nhóm thuốc 14
4.3 Bảng Suppliers – nhà cung cấp 14
4.4 Bảng Medicines – danh mục thuốc 15
4.5 Bảng Batches – lô thuốc (trung tâm của FEFO) 15
4.6 Bảng Invoices – hóa đơn bán 15
4.7 Bảng InvoiceLines – chi tiết hóa đơn 16
4.8 Bảng ReceiptNotes và ReceiptLines – phiếu nhập và chi tiết 16
4.9 Các ràng buộc toàn vẹn chính 17
4.10 Các tình huống có sẵn trong dữ liệu mẫu 18

5.1 Trách nhiệm và ví dụ của từng lớp 20
5.2 Các ngoại lệ nghiệp vụ 22

8.1 Môi trường kiểm thử 44
8.2 Bảng ca kiểm thử (20 ca) 45
8.3 Tổng hợp kết quả kiểm thử 46
8.4 Lỗi đã phát hiện và cách khắc phục 47

A.1 Tài khoản demo 51

B.1 Phân công công việc 53

Danh mục từ viết tắt

Từ viết tắt	Ý nghĩa
MVP	Model – View – Presenter
BLL / DAL	Business Logic Layer (tầng nghiệp vụ) / Data Access Layer (tầng truy cập dữ liệu)
FEFO	First Expired, First Out – lô hết hạn trước thì xuất trước
CSDL	Cơ sở dữ liệu
ERD	Entity Relationship Diagram – sơ đồ thực thể quan hệ
PK / FK	Primary Key (khóa chính) / Foreign Key (khóa ngoại)
T-SQL	Transact-SQL, ngôn ngữ truy vấn mở rộng của SQL Server
ADO.NET	Bộ thư viện truy cập dữ liệu của .NET
MDI	Multiple Document Interface – giao diện nhiều cửa sổ con
PBKDF2	Password-Based Key Derivation Function 2
UI / UX	Giao diện người dùng / Trải nghiệm người dùng
TC	Test Case – ca kiểm thử

Chương 1

Giới thiệu bài toán và nhu cầu thực tế

1.1 Đặt vấn đề

Nhà thuốc là nơi mỗi sai sót về quản lý kho đều có thể ảnh hưởng trực tiếp đến sức khỏe người dùng. Một hộp thuốc quá hạn bị bán ra, hoặc một lô thuốc sắp hết hạn bị bỏ quên trong kho, đều gây thiệt hại cho cửa hàng và rủi ro cho khách hàng. Khi quản lý bằng sổ sách hoặc bảng tính, người bán thường gặp các khó khăn sau:

- Cùng một loại thuốc nhưng nhập nhiều lần, mỗi lần có *hạn sử dụng* khác nhau; khó biết nên bán lô nào trước.
- Không có cảnh báo tự động cho thuốc đã hết hạn, sắp hết hạn hoặc sắp hết hàng.
- Dễ sai lệch tồn kho khi bán và nhập kho diễn ra liên tục, nhất là khi có nhiều quầy cùng thao tác.
- Hóa đơn, báo cáo doanh thu phải tổng hợp thủ công, mất thời gian và dễ nhầm lẫn.

Đề tài Đề tài 14 – *Quản lý nhà thuốc* – xây dựng ứng dụng desktop **PharmaCare** nhằm giải quyết các vấn đề trên bằng cách quản lý thuốc theo từng *lô*, tự động chọn lô bán theo nguyên tắc FEFO và cảnh báo hạn dùng.

1.2 Mục tiêu đề tài

1.2.1 Mục tiêu chung

Xây dựng ứng dụng Windows Forms trên nền .NET kết nối SQL Server, hỗ trợ nhà thuốc quản lý danh mục thuốc, nhập kho theo lô, bán hàng, lập và in hóa đơn, theo dõi hạn dùng và thống kê doanh thu.

1.2.2 Mục tiêu cụ thể

1. Phân tích nghiệp vụ nhà thuốc và đặc tả yêu cầu chức năng, phi chức năng.
2. Thiết kế cơ sở dữ liệu quan hệ có đầy đủ ràng buộc toàn vẹn và thủ tục lưu sẵn.
3. Thiết kế kiến trúc phần mềm theo mô hình MVP kết hợp nhiều lớp (View – Presenter – Service – Repository) để dễ bảo trì và kiểm thử.
4. Cài đặt thuật toán bán theo FEFO, giao dịch bán/nhập kho an toàn (có rollback), phân quyền theo vai trò.
5. Kiểm thử chức năng bằng bộ ca kiểm thử có cả tình huống lỗi, ghi nhận kết quả và khắc phục lỗi phát hiện được.

1.3 Phạm vi chức năng và đối tượng sử dụng

Hệ thống phục vụ một nhà thuốc quy mô nhỏ, chạy trên máy Windows và dùng một cơ sở dữ liệu SQL Server cục bộ. Có hai nhóm người dùng:

- **Dược sĩ bán hàng**: tra cứu danh mục, bán hàng, in hóa đơn, xem hóa đơn và cảnh báo hạn dùng.
- **Quản lý kho**: quản lý danh mục thuốc, nhập kho theo lô, xem hóa đơn, cảnh báo và báo cáo doanh thu.

Ngoài phạm vi: thanh toán điện tử, kết nối máy quét mã vạch, quản lý nhiều chi nhánh, ứng dụng web/di động.

1.4 Phương pháp thực hiện

Nhóm làm việc theo các bước: (1) khảo sát nghiệp vụ và xác định yêu cầu; (2) thiết kế CSDL và kiến trúc; (3) cài đặt theo từng chức năng, mỗi thành viên phụ trách một nhóm module và đẩy mã lên kho GitHub chung; (4) kiểm thử bằng bộ ca kiểm thử, rà soát mã nguồn và sửa lỗi; (5) hoàn thiện tài liệu. Phần phân công chi tiết xem Phụ lục B.

1.5 Công cụ và môi trường

1.6 Bố cục báo cáo

Báo cáo gồm chín chương: Chương 1 giới thiệu bài toán và nhu cầu thực tế; Chương 2 trình bày cơ sở lý thuyết; Chương 3 phân tích người dùng, vai trò và phạm vi chức năng; Chương 4 thiết kế cơ sở dữ liệu; Chương 5 trình bày kiến trúc chương trình; Chương 6 mô tả các chức năng chính;

Bảng 1.1: Công cụ và môi trường phát triển

Hạng mục	**Công cụ / phiên bản**
Ngôn ngữ, nền tảng	C#, .NET 10, Windows Forms
Cơ sở dữ liệu	SQL Server (Express/Developer), SSMS
Thư viện truy cập dữ liệu	Microsoft.Data.SqlClient (ADO.NET)
IDE	Visual Studio 2026
Quản lý mã nguồn	Git, GitHub
Soạn báo cáo	LaTeX (XeLaTeX), Overleaf
Vẽ sơ đồ	TikZ (ERD, kiến trúc), PlantUML (use case, class, sequence)

Chương 7 trình bày thiết kế giao diện; Chương 8 trình bày kiểm thử và kết quả; Chương 9 nêu kết luận, hạn chế và hướng phát triển. Phần phụ lục gồm hướng dẫn cài đặt, tài khoản demo và script cơ sở dữ liệu, bảng đóng góp của thành viên và mã nguồn tiêu biểu.

Chương 2

Cơ sở lý thuyết

2.1 .NET và ngôn ngữ C#

.NET là nền tảng phát triển ứng dụng của Microsoft, cung cấp môi trường chạy, thư viện lớp cơ sở và công cụ biên dịch; C# là ngôn ngữ hướng đối tượng được dùng phổ biến trên nền tảng này [1]. Trong đề tài, toàn bộ mã nguồn được viết bằng C# và chạy trên .NET. Một số tính năng của C# được dùng nhiều trong đề tài. *Kiểu tham chiếu và kiểu giá trị*: lớp như `Thuoc` là kiểu tham chiếu, còn `record` như `DongNhap` là kiểu dữ liệu bất biến gọn cho một dòng nhập. *Generic* cho phép dùng cùng một cấu trúc với nhiều kiểu mà vẫn kiểm tra kiểu lúc biên dịch, ví dụ `List<T>`, `IReadOnlyList<ChiTietHoaDon>`, `HashSet<Quyen>`. *LINQ* viết truy vấn trên tập hợp ngay trong C#, như `Dong.Sum(...)` để tính tổng phiếu nhập hay `Rows.Cast<DataRow>().Take(8)` để lấy 8 dòng đầu. *Nullable reference types* (bật bằng `<Nullable>enable</Nullable>` trong tệp dự án) buộc khai báo rõ giá trị nào có thể rỗng: `Thuoc?` nghĩa là có thể không tìm thấy thuốc, còn `string` thì không được rỗng, nhờ đó trình biên dịch cảnh báo sớm lỗi tham chiếu rỗng. Cuối cùng, *event* và *delegate* hiện thực mẫu quan sát: `GioHang` phát sự kiện `DaThayDoi` mỗi khi thêm hoặc bỏ dòng để Presenter cập nhật View, còn `AppEvents.KhoThayDoi` thông báo toàn ứng dụng khi kho đổi để các trang đang mở tự tải lại. Bên phát sự kiện không cần biết ai đang lắng nghe.

2.2 Lập trình hướng đối tượng

Đề tài áp dụng bốn tính chất của lập trình hướng đối tượng: *đóng gói* (thuộc tính có kiểm tra hợp lệ trong lớp `Nguoi`), *kế thừa* (`DuocSi` và `QuanLyKho` kế thừa `Nguoi`), *đa hình* (mỗi vai trò tự khai báo tập quyền) và *trừu tượng hóa* qua giao diện (`interface`) cho View và Repository. *Lớp trừu tượng và giao diện* khác nhau ở chỗ lớp trừu tượng có thể chứa trạng thái và mã dùng chung còn giao diện chỉ là hợp đồng, và một lớp chỉ kế thừa một lớp cha nhưng cài đặt được nhiều giao diện. `Nguoi` là lớp trừu tượng vì `DuocSi` và `QuanLyKho` chia sẻ dữ liệu (tên đăng nhập, họ tên có kiểm tra hợp lệ) và chỉ khác nhau ở thành viên trừu tượng khai báo tập quyền; ngược lại

IKhoRepository hay INhomThuocView chỉ mô tả hành vi mà không ràng buộc cách cài đặt. Đề tài cũng bám theo các nguyên lý SOLID: mỗi lớp một trách nhiệm (View hiển thị, Presenter điều phối, Service giữ luật nghiệp vụ, Repository truy cập dữ liệu); thêm một vai trò mới chỉ cần thêm lớp con của Nguoi mà không sửa mã cũ; DuocSi và QuanLyKho dùng thay cho Nguoi ở mọi nơi; mỗi màn hình có giao diện View riêng nhỏ gọn thay vì một giao diện lớn chung; và Service nhận giao diện Repository qua hàm khởi tạo nên phụ thuộc vào trừu tượng chứ không vào lớp cụ thể [2].

2.3 Windows Forms

Windows Forms là bộ thư viện dựng giao diện desktop trên Windows [3]. Ứng dụng dùng mô hình *MDI* (form cha chứa các form con), ErrorProvider để báo lỗi tại ô nhập, BindingSource để lọc dữ liệu nhanh trên bộ nhớ, ContextMenuStrip cho menu chuột phải và PrintDocument/PrintPreviewDialog để in hóa đơn. Windows Forms theo mô hình hướng sự kiện: chương trình chạy vòng lặp thông điệp và gọi các hàm xử lý khi người dùng thao tác. Một form đi qua các sự kiện Load (trước khi hiện lần đầu), Shown (đã hiện xong), FormClosing (có thể hủy việc đóng, đề tài dùng để hỏi xác nhận thoát) và FormClosed. *Data binding* gắn nguồn dữ liệu vào điều khiển, ví dụ gán DataTable cho DataGridView.DataSource; BindingSource.Filter lọc trên dữ liệu đã nạp nên không cần truy vấn lại SQL. Với *MDI*, form cha đặt IsMdiContainer=true và mỗi trang là form con có MdiParent; MdiChildren cho phép duyệt các trang đang mở để chỉ kích hoạt lại thay vì mở trùng. Việc *in ấn* dựa trên PrintDocument: sự kiện BeginPrint đặt lại trạng thái, PrintPage vẽ từng trang bằng Graphics và đặt HasMorePages để sang trang tiếp, còn PrintPreviewDialog dùng cùng đường vẽ đó để xem trước. Đề tài vẽ hóa đơn bằng một hàm dùng chung cho cả xem trước và in thật nên hai kết quả luôn giống nhau.

2.4 SQL Server và T-SQL

SQL Server là hệ quản trị cơ sở dữ liệu quan hệ của Microsoft [4]. Các khái niệm được dùng trong đề tài gồm: khóa chính/khóa ngoại, ràng buộc CHECK/UNIQUE, chỉ mục, thủ tục lưu sẵn (stored procedure), giao dịch và khóa (UPDLOCK, HOLDLOCK) [5]. *Chuẩn hóa* giảm dư thừa dữ liệu: dạng chuẩn 1 yêu cầu mỗi ô chứa một giá trị nguyên tử; dạng chuẩn 2 yêu cầu thuộc tính không khóa phụ thuộc toàn bộ khóa; dạng chuẩn 3 loại bỏ phụ thuộc bắc cầu [5]. Lược đồ của đề tài tách Categories, Suppliers, Medicines, Batches và các bảng dòng chi tiết thay vì lưu tên nhóm hay tên nhà cung cấp lặp lại ở từng dòng, và tách riêng Batches vì một thuốc có nhiều lô với hạn dùng và tồn khác nhau. *Chỉ mục* tăng tốc truy vấn đổi lại chi phí ghi: IX_Batches_Medicine_Expiry theo (thuốc, hạn dùng) phục vụ việc chọn lô theo FEFO. *Mức cô lập* quyết định giao dịch thấy dữ liệu chưa commit của nhau đến đâu; SQL Server mặc định dùng READCOMMITTED. *Stored procedure* đặt logic gần dữ liệu và dùng lại được: sp_Lo_Nhap kiểm tra dữ liệu,

đọc lô theo mã với UPDLOCK,HOLDLOCK để hai phiên nhập cùng một mã lô không thể cùng chèn, rồi cộng dồn hoặc thêm lô; lỗi nghiệp vụ được phát bằng THROW với mã riêng (50010–50014) và lớp Repository ánh xạ sang ngoại lệ tương ứng ở C#.

2.5 ADO.NET và phòng chống SQL Injection

ADO.NET cung cấp các lớp SqlConnection, SqlCommand, SqlParameter, SqlTransaction [6]. Mọi truy vấn của đề tài truyền giá trị qua SqlParameter thay vì nối chuỗi, nhờ đó chuỗi như ' OR 1=1 -- chỉ được coi là dữ liệu, không thể thay đổi câu lệnh SQL. *SQL injection* xảy ra khi dữ liệu người dùng được nối vào chuỗi lệnh SQL nên một phần dữ liệu bị hiểu như lệnh. *Tham số hóa* ngăn điều này vì câu lệnh và giá trị được gửi tách nhau: máy chủ phân tích cú pháp câu lệnh trước, giá trị tham số luôn chỉ là dữ liệu, dù chứa dấu nháy hay từ khóa. Đề tài truyền mọi giá trị qua SqlParameter. Ký tự đại diện của LIKE (%, _, []) là vấn đề khác: chúng không phá câu lệnh nhưng làm thay đổi kết quả tìm kiếm, nên được thoát bằng hàm EscapeLike để người dùng gõ % được hiểu như ký tự thường. Về *connection pooling*, thư viện giữ lại các kết nối đã mở để dùng lại khi có cùng chuỗi kết nối nên việc mở kết nối ngắn hạn rất rẻ; vì vậy mã của đề tài mở kết nối trong khối using cho từng thao tác và để Dispose trả kết nối về nhóm.

2.6 Mô hình MVP và kiến trúc nhiều lớp

MVP (Model–View–Presenter) tách giao diện (View) khỏi logic điều phối (Presenter); View chỉ hiển thị và phát sinh sự kiện, Presenter xử lý và yêu cầu View cập nhật qua giao diện [7, 8]. Kết hợp với tầng Service (nghiệp vụ) và Repository (truy cập dữ liệu), mỗi lớp chỉ phụ thuộc lớp bên dưới thông qua giao diện, giúp dễ thay thế và kiểm thử [2]. Ba mẫu thường gặp cùng mục tiêu tách giao diện khỏi logic. Trong *MVC*, Controller nhận thao tác đầu vào còn View có thể đọc trực tiếp Model. Trong *MVP*, Presenter đứng giữa và View không biết đến Model; View và Presenter nói chuyện qua giao diện. Trong *MVVM*, View gắn kết hai chiều với ViewModel nhờ cơ chế binding mạnh, phù hợp WPF hơn WinForms. Fowler phân MVP thành *Passive View*, mọi logic nằm ở Presenter còn View chỉ có hàm đọc và ghi dữ liệu hiển thị, và *Supervising Controller*, để View tự lo các binding đơn giản [8]. Đề tài gần với Passive View: View trả về chuỗi thuần, Presenter tự TryParse, kiểm tra và bắt ngoại lệ. Ưu điểm là Presenter kiểm thử được bằng View giả mà không cần mở form, và View không chứa SQL hay luật nghiệp vụ; nhược điểm là phải viết thêm giao diện và đăng ký sự kiện cho từng màn hình, và WinForms không có binding hai chiều mạnh như MVVM nên có thêm mã nối.

2.7 Giao dịch cơ sở dữ liệu và tính ACID

Giao dịch là một nhóm thao tác được thực hiện trọn vẹn hoặc không thực hiện gì (nguyên tử). Khi bán hàng, việc ghi hóa đơn và trừ tồn kho từng lô phải nằm trong cùng một giao dịch để không xảy ra hóa đơn mồ côi hoặc tồn kho âm. Bốn tính chất ACID [5]: *Atomicity* (nguyên tử): giao dịch xong trọn vẹn hoặc không để lại gì; *Consistency* (nhất quán): dữ liệu luôn thỏa các ràng buộc; *Isolation* (cô lập): các giao dịch đồng thời không làm hỏng nhau; *Durability* (bền vững): dữ liệu đã commit không mất. Trong đề tài, tính nguyên tử có được nhờ `SqlTransaction`: lỗi ở bất kỳ dòng nào thì `Rollback` cả hóa đơn lẫn phần tồn đã trừ; tính nhất quán nhờ ràng buộc `CHECK` và khóa ngoại; tính bền vững do SQL Server ghi nhật ký giao dịch. Về cô lập, việc trừ kho dùng câu lệnh `UPDATE` có điều kiện `WHERE QuantityRemaining >= @sl` (và lô chưa hết hạn). Khi hai quầy cùng bán lô còn ít hàng, SQL Server khóa dòng lúc cập nhật nên một quầy thành công, quầy kia thấy 0 dòng bị ảnh hưởng và giao dịch được hoàn tác. Đây là cách *lạc quan* (optimistic): không khóa trước mà kiểm tra điều kiện ngay khi ghi, khác với cách *bi quan* (pessimistic) khóa dòng ngay khi đọc như `UPDLOCK` trong `sp_Lo_Nhap`. *Deadlock* là khi hai giao dịch giữ khóa và chờ khóa của nhau, SQL Server chọn một giao dịch làm nạn nhân; đề tài giảm nguy cơ này bằng cách giữ giao dịch ngắn và không chờ thao tác của người dùng ở bên trong giao dịch.

2.8 Phương pháp quản lý kho FEFO

FEFO (First Expired, First Out) ưu tiên xuất lô có hạn dùng gần nhất, khác với FIFO xuất theo thứ tự nhập. Với thuốc, FEFO giảm nguy cơ tồn đọng và hết hạn trong kho. Ba chiến lược xuất kho: *FIFO* xuất lô nhập trước, *LIFO* xuất lô nhập sau, còn *FEFO* xuất lô có hạn dùng gần nhất. Hai lô nhập cùng thời điểm có thể có hạn rất khác nhau, hoặc lô nhập sau lại có hạn ngắn hơn, nên FIFO không bảo đảm bán hết thuốc sắp hết hạn trước; vì thuốc hết hạn không được bán, FEFO là lựa chọn phù hợp. Trong đề tài, `BanHangService.PhanBoFefo` sắp các lô còn tồn theo hạn dùng, bỏ lô đã hết hạn, lấy từ lô gần hạn nhất và chuyển sang lô kế tiếp khi chưa đủ số lượng, có tính cả phần đã nằm trong giỏ hàng.

2.9 Bảo mật mật khẩu

Mật khẩu không được lưu dạng thô mà được băm một chiều cùng giá trị *salt* ngẫu nhiên. PBKDF2 là hàm dẫn xuất khóa chạy nhiều vòng lặp để làm chậm tấn công vét cạn [9, 10]. Đề tài dùng PBKDF2-SHA256 với salt 16 byte và 100 000 vòng lặp, so sánh kết quả bằng `FixedTimeEquals` để tránh rò rỉ thông tin qua thời gian xử lý. *Mã hóa* là hai chiều (có khóa thì giải ngược được) còn *băm* là một chiều, nên lưu mật khẩu cần băm để kẻ lấy được cơ sở dữ liệu không đọc lại được mật khẩu. *Salt* là chuỗi ngẫu nhiên riêng cho từng mật khẩu, trộn vào trước khi băm: hai người cùng mật khẩu vẫn có giá trị băm khác nhau và bảng tính sẵn (rainbow table) trở nên vô dụng.

Work factor (số vòng lặp) làm mỗi lần băm tốn thời gian: không đáng kể với người dùng đăng nhập một lần nhưng làm tấn công thử hàng loạt chậm đi rất nhiều [9, 10]. Giá trị lưu có dạng `PBKDF2$vòng$salt$hash` nên số vòng có thể nâng lên về sau mà các tài khoản cũ vẫn kiểm tra được. *Timing attack* suy ra thông tin từ thời gian so sánh: so sánh thông thường dừng ở byte đầu tiên khác nhau, còn `FixedTimeEquals` luôn mất thời gian như nhau nên không rò rỉ gì.

Chương 3

Phân tích người dùng, vai trò và phạm vi chức năng

3.1 Khảo sát nghiệp vụ

Quy trình điển hình của một nhà thuốc nhỏ gồm: (1) nhập thuốc từ nhà cung cấp theo từng lô, ghi hạn sử dụng; (2) dược sĩ bán thuốc cho khách, thuốc kê đơn cần chú ý; (3) kiểm tra định kỳ các lô sắp hết hạn để xử lý; (4) cuối ngày/tháng tổng hợp doanh thu. Từ đó nhóm rút ra các nhu cầu cốt lõi: quản lý theo *lô*, bán theo hạn dùng, cảnh báo, hóa đơn và báo cáo.

3.2 Tác nhân và phân quyền

Hệ thống có hai tác nhân. Quyền được định nghĩa trong lớp `Nguoi` (kế thừa bởi `DuocSi` và `QuanLyKho`); Bảng 3.1 là ma trận phân quyền.

Bảng 3.1: Ma trận phân quyền theo vai trò

Quyền / chức năng	Dược sĩ	Quản lý kho
Xem danh mục thuốc, nhóm thuốc, nhà cung cấp	✓	✓
Thêm / sửa / xóa thuốc	–	✓
Đổi trạng thái kê đơn	✓	✓
Nhập kho theo lô	–	✓
Xem / tra cứu lịch sử phiếu nhập	–	✓
Bán hàng, in hóa đơn	✓	–
Xem / tra cứu hóa đơn	✓	✓
Xem cảnh báo hạn dùng	✓	✓
Báo cáo doanh thu	–	✓

3.3 Yêu cầu chức năng

Bảng 3.2: Danh sách yêu cầu chức năng

Mã	Chức năng	Mô tả
FR01	Đăng nhập	Xác thực bằng tên đăng nhập, mật khẩu; tài khoản bị khóa không đăng nhập được; mở giao diện theo vai trò.
FR02	Quản lý thuốc	Xem, tìm nhanh, sắp xếp; thêm, sửa, xóa thuốc; gắn ảnh; kiểm tra dữ liệu nhập.
FR03	Nhóm thuốc, nhà cung cấp	Xem danh sách; đổi trạng thái kê đơn của thuốc theo nhóm.
FR04	Nhập kho theo lô	Ghi nhận lô (mã lô, hạn dùng, số lượng, giá nhập, giá bán); nhập lại lô đã có thì cộng dồn.
FR05	Bán hàng FEFO	Chọn thuốc và số lượng, hệ thống tự phân bổ từ lô có hạn gần nhất, không lấy lô hết hạn, không bán vượt tồn.
FR06	Thanh toán	Lưu hóa đơn và trừ tồn kho trong một giao dịch; lỗi thì hoàn tác toàn bộ.
FR07	In hóa đơn	Xem trước và in hóa đơn, tự sang trang khi nhiều dòng.
FR08	Tra cứu hóa đơn	Lọc theo từ khóa và khoảng ngày; xem chi tiết từng hóa đơn.
FR09	Cảnh báo hạn dùng	Phân loại đã hết hạn, sắp hết hạn, sắp hết hàng; tô màu; tự cập nhật khi kho thay đổi.
FR10	Báo cáo doanh thu	Tổng hợp doanh thu theo ngày cho quản lý.
FR11	Tổng quan	Trang tổng quan với số liệu chính và các mục cần chú ý.
FR12	Tra cứu phiếu nhập	Lọc phiếu nhập theo khoảng ngày, nhà cung cấp và từ khóa (mã phiếu, người lập); xem chi tiết các lô của từng phiếu; chỉ Quản lý kho.

3.4 Yêu cầu phi chức năng

- **Toàn vẹn dữ liệu**: ràng buộc `CHECK`, `UNIQUE`, khóa ngoại ở mức CSDL; giao dịch có rollback.
- **Bảo mật**: mật khẩu băm PBKDF2-SHA256; truy vấn tham số hóa; kiểm tra quyền ở cả giao diện và tầng Service.
- **Dễ bảo trì**: kiến trúc MVP kết hợp nhiều lớp, mỗi lớp một trách nhiệm, dùng giao diện để tách phụ thuộc.
- **Dễ triển khai**: ứng dụng tự dò máy chủ SQL Server và tự tạo cơ sở dữ liệu mẫu khi chạy lần đầu.

- **Khả năng sử dụng**: báo lỗi nhập liệu ngay tại ô; thông báo bằng tiếng Việt, nêu rõ cách khắc phục.

3.5 Quy tắc nghiệp vụ

BR1. Mã thuốc và mã lô là duy nhất.

BR2. Giá bán phải lớn hơn 0; số lượng nhập, số lượng bán phải lớn hơn 0.

BR3. Không nhập lô có hạn dùng là hôm nay hoặc đã qua; hạn dùng không được trước ngày nhập.

BR4. Nhập lại một mã lô đã có phải giữ đúng hạn dùng của lô đó; số lượng được cộng dồn, *giá bán của lô giữ nguyên*, giá nhập tính bình quân gia quyền.

BR5. Khi bán: chỉ lấy lô còn hạn, ưu tiên hạn gần nhất, có xét phần đã nằm trong giỏ hàng.

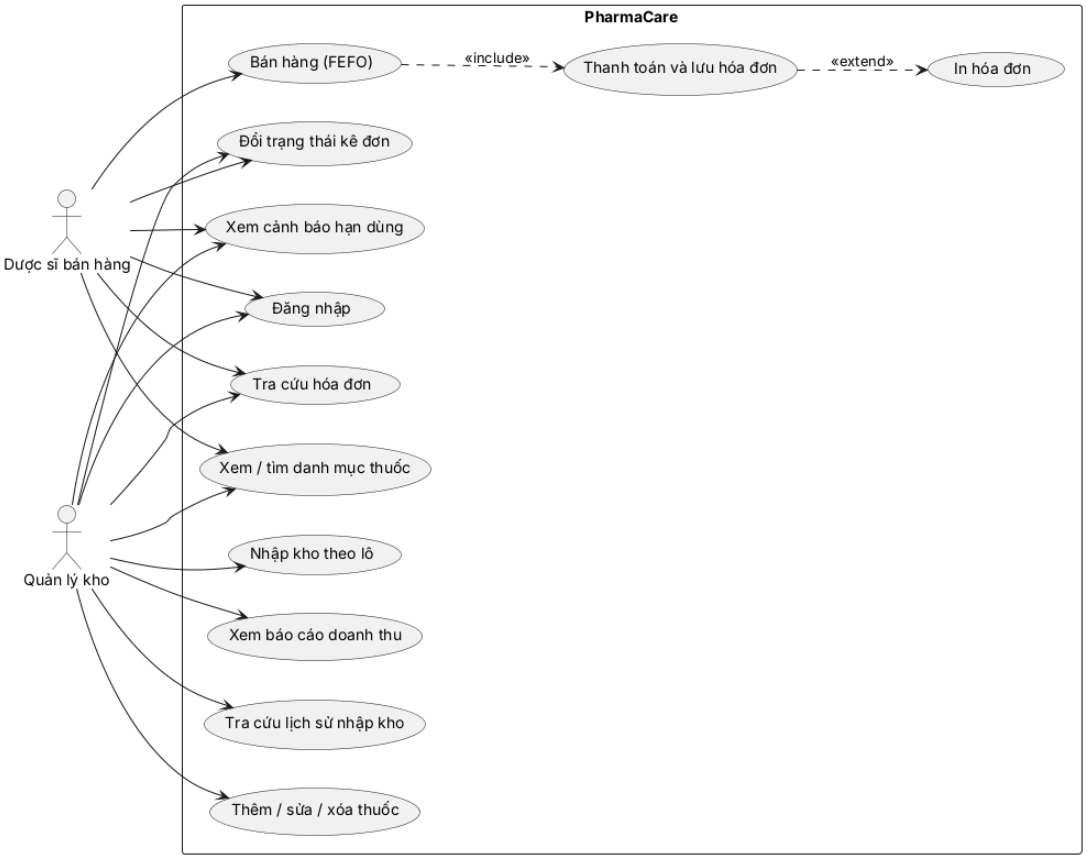
BR6. Một hóa đơn được lưu trọn vẹn hoặc không lưu gì; tổng tiền bằng tổng (số lượng $\times$ đơn giá) các dòng.

BR7. Giá dùng khi bán là giá của từng lô; giá niêm yết của thuốc chỉ dùng để điền sẵn khi nhập lô.

BR8. Mỗi lần nhập kho tạo một phiếu nhập ghi người lập, nhà cung cấp, ngày và tổng tiền bằng tổng (số lượng $\times$ giá nhập) các dòng; lịch sử phiếu nhập chỉ Quản lý kho được xem.

BR9. Cảnh báo: đã hết hạn (số ngày còn lại < 0), sắp hết hạn (≤ 30 ngày), sắp hết hàng (tồn ≤ 10).

3.6 Mô hình ca sử dụng



Hình 3.1: Sơ đồ ca sử dụng (Use Case) của hệ thống PharmaCare

Chương 4

Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu `PharmaCareDB` trên SQL Server gồm chín bảng, được tạo bằng ba script đặt trong thư mục `Database/`: `01_CreateDatabase.sql` (bảng, ràng buộc, chỉ mục), `02_InsertSampleData.sql` (dữ liệu mẫu) và `03_StoredProcedures.sql` (thủ tục lưu sẵn). Script thứ tư `04_BackupDatabase.sql` dùng để tạo tệp sao lưu `.bak`.

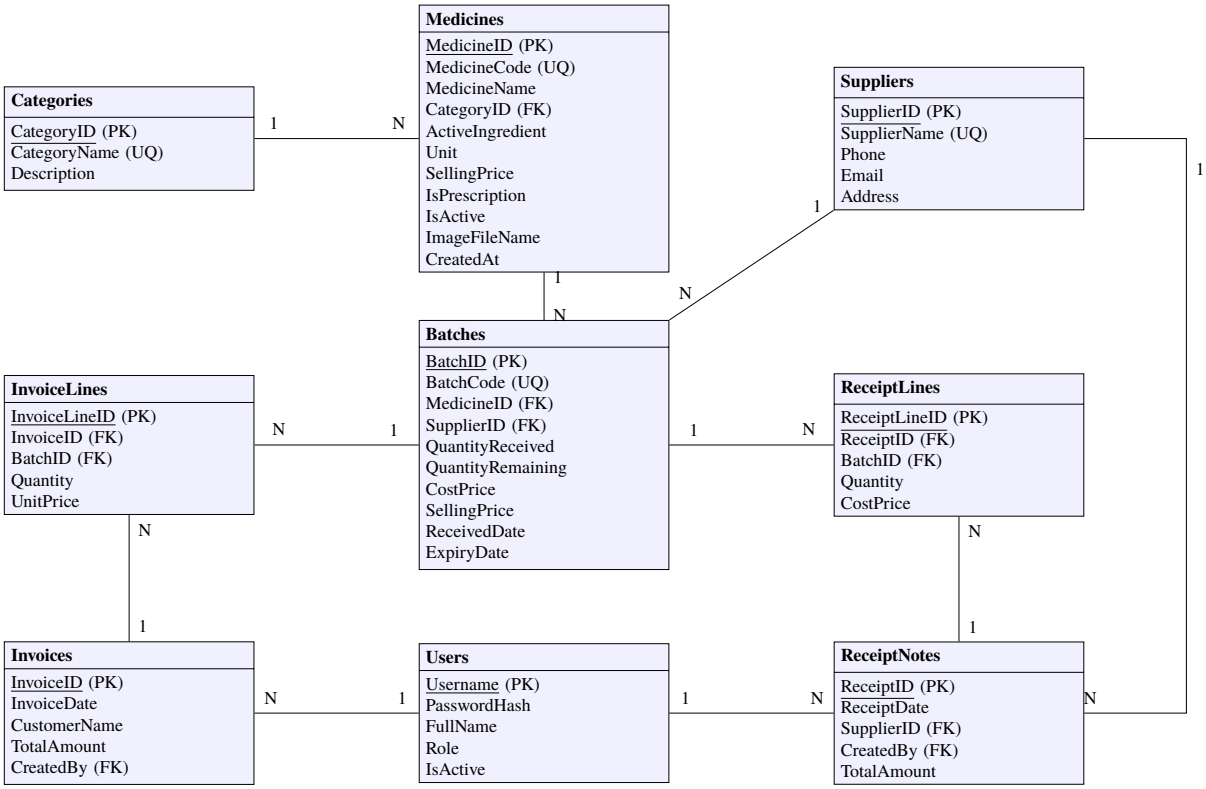
4.1 Mô hình thực thể quan hệ

Mỗi *thuốc* (Medicines) thuộc một *nhóm thuốc* (Categories) và có nhiều *lô* (Batches); mỗi lô có hạn dùng và số lượng còn lại riêng. Lô được cung cấp bởi *nhà cung cấp* (Suppliers). *Hóa đơn* (Invoices) gồm nhiều *dòng hóa đơn* (InvoiceLines), mỗi dòng trỏ tới đúng một lô đã xuất; tương tự *phiếu nhập* (ReceiptNotes) gồm nhiều *dòng phiếu nhập* (ReceiptLines) trỏ tới lô đã nhập. Hóa đơn và phiếu nhập lưu người lập (Users). Thiết kế quản lý tồn kho theo *lô* thay vì theo thuốc là điểm mấu chốt cho phép bán theo FEFO.

4.2 Mô tả chi tiết các bảng

Bảng 4.1: Bảng Users – tài khoản đăng nhập

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
Username	VARCHAR(50)	PK	Tên đăng nhập
PasswordHash	VARCHAR(200)	NOT NULL	Chuỗi băm PBKDF2-SHA256 (có salt), không lưu mật khẩu thô
FullName	NVARCHAR(100)	NOT NULL	Họ tên hiển thị
Role	VARCHAR(20)	CHECK IN (DUOCSI, QUANLYKHO)	Vai trò
IsActive	BIT	DEFAULT 1	0 = tài khoản bị khóa



Hình 4.1: Sơ đồ thực thể quan hệ (ERD) của PharmaCareDB

Bảng 4.2: Bảng Categories – nhóm thuốc

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
CategoryID	INT IDENTITY	PK	Mã nhóm
CategoryName	NVARCHAR(100)	NOT NULL, UNIQUE	Tên nhóm
Description	NVARCHAR(300)	NULL	Mô tả

Bảng 4.3: Bảng Suppliers – nhà cung cấp

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
SupplierID	INT IDENTITY	PK	Mã nhà cung cấp
SupplierName	NVARCHAR(160)	NOT NULL, UNIQUE	Tên nhà cung cấp
Phone, Email	VARCHAR	NULL	Liên hệ
Address	NVARCHAR(250)	NULL	Địa chỉ

Bảng 4.4: Bảng Medicines – danh mục thuốc

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
MedicineID	INT IDENTITY	PK	Mã nội bộ
MedicineCode	VARCHAR(30)	NOT NULL, UNIQUE	Mã thuốc (ví dụ MED001)
MedicineName	NVARCHAR(160)	NOT NULL	Tên thuốc
CategoryID	INT	FK → Categories	Nhóm thuốc
ActiveIngredient	NVARCHAR(160)	NULL	Hoạt chất
Unit	NVARCHAR(30)	DEFAULT ‘Viên’	Đơn vị tính
SellingPrice	DECIMAL(18,2)	CHECK > 0	Giá niêm yết (điền sẵn khi nhập lô)
IsPrescription	BIT	DEFAULT 0	Thuốc kê đơn
IsActive	BIT	DEFAULT 1	Còn kinh doanh
ImageFileName	NVARCHAR(260)	NULL	Tên tệp ảnh (ảnh nằm ở thư mục Images)
CreatedAt	DATETIME2	DEFAULT SYSDATETIME()	Thời điểm tạo

Bảng 4.5: Bảng Batches – lô thuốc (trung tâm của FEFO)

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
BatchID	INT IDENTITY	PK	Mã lô nội bộ
BatchCode	VARCHAR(40)	NOT NULL, UNIQUE	Mã lô ghi trên bao bì
MedicineID	INT	FK → Medicines	Thuốc của lô
SupplierID	INT	FK → Suppliers, NULL	Nhà cung cấp
QuantityReceived	INT	CHECK ≥ 0	Tổng số lượng đã nhập
QuantityRemaining	INT	CHECK ≥ 0	Số lượng còn trong kho
CostPrice	DECIMAL(18,2)	CHECK ≥ 0	Giá nhập
SellingPrice	DECIMAL(18,2)	CHECK > 0	Giá bán của lô
ReceivedDate	DATE	NOT NULL	Ngày nhập
ExpiryDate	DATE	CHECK $\geq$ ReceivedDate	Hạn sử dụng

Bảng 4.6: Bảng Invoices – hóa đơn bán

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
InvoiceID	INT IDENTITY	PK	Số hóa đơn
InvoiceDate	DATETIME2	DEFAULT SYSDATETIME()	Thời điểm lập
CustomerName	NVARCHAR(160)	DEFAULT ‘Khách lẻ’	Tên khách
TotalAmount	DECIMAL(18,2)	CHECK ≥ 0	Tổng tiền
CreatedBy	VARCHAR(50)	FK → Users	Người lập

Bảng 4.7: Bảng InvoiceLines – chi tiết hóa đơn

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
InvoiceLineID	INT IDENTITY	PK	Mã dòng
InvoiceID	INT	FK $\rightarrow$ Invoices	Hóa đơn
BatchID	INT	FK $\rightarrow$ Batches	Lô đã xuất
Quantity	INT	CHECK > 0	Số lượng bán
UnitPrice	DECIMAL(18,2)	CHECK ≥ 0	Đơn giá tại thời điểm bán

Bảng 4.8: Bảng ReceiptNotes và ReceiptLines – phiếu nhập và chi tiết

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
ReceiptID	INT IDENTITY	PK (ReceiptNotes)	Số phiếu nhập
ReceiptDate, TotalAmount	DATETIME2, DECIMAL	DEFAULT, CHECK ≥ 0	Ngày và tổng tiền nhập
SupplierID, CreatedBy	INT, VARCHAR	FK $\rightarrow$ Suppliers, Users	Nhà cung cấp, người lập
ReceiptLineID	INT IDENTITY	PK (ReceiptLines)	Mã dòng
ReceiptID, BatchID	INT	FK	Phiếu và lô tương ứng
Quantity, CostPrice	INT, DECIMAL	CHECK > 0, ≥ 0	Số lượng và giá nhập của dòng

4.3 Ràng buộc toàn vẹn và chỉ mục

Ngoài khóa chính, khóa ngoại, các ràng buộc quan trọng được đặt ngay ở CSDL để dữ liệu luôn đúng dù ứng dụng có lỗi (Bảng 4.9). Hai chỉ mục được tạo: `IX_Batches_Medicine_Expiry` trên `(MedicineID, ExpiryDate)` *include* `QuantityRemaining, SellingPrice` phục vụ truy vấn chọn lô theo FEFO và cảnh báo hạn dùng; `IX_Invoices_Date` trên `InvoiceDate` phục vụ tra cứu hóa đơn và báo cáo theo ngày.

4.4 Thủ tục lưu sẵn

`sp_Lo_CanhBao(@DaysAhead=30, @LowStock=10)` trả về các lô có tồn > 0 kèm trạng thái: *Đã hết hạn*, *Sắp hết hạn* (≤ 30 ngày) hoặc *Sắp hết hàng* (tồn ≤ 10) và số ngày còn lại; sắp xếp theo hạn dùng. Dùng cho trang Cảnh báo và Tổng quan.

`sp_Lo_TonTheoLo(@MedicineID=NULL)` liệt kê tồn kho theo từng lô (của một thuốc hoặc tất cả), dùng cho trang Danh mục thuốc và Nhập kho.

`sp_Lo_Nhap` nhập lô theo kiểu *upsert*: mã lô chưa có thì thêm mới, đã có thì cộng dồn số lượng (Danh sách mã 4.1).

Danh sách mã 4.1: Thủ tục `sp_Lo_Nhap` (trích từ `03_StoredProcedures.sql`)

```
37 CREATE OR ALTER PROCEDURE dbo.sp_Lo_Nhap
```

Bảng 4.9: Các ràng buộc toàn vẹn chính

Ràng buộc	Bảng	Mục đích
UNIQUE	MedicineCode, BatchCode, CategoryName, SupplierName	Không trùng mã / tên
CHECK (> 0)	SellingPrice (Medicines, Batches), Quantity (lines)	Không có giá hoặc số lượng không hợp lệ
CHECK (≥ 0)	QuantityRemaining, QuantityReceived, CostPrice, TotalAmount	Tồn kho không bao giờ âm
CK_Batches_Expiry	Batches	Hạn dùng không trước ngày nhập
CK_Users_Role	Users	Chỉ nhận hai vai trò hợp lệ
FOREIGN KEY	Medicines, Batches, lines, Invoices, ReceiptNotes	Không xóa thuốc đã có lô; không có dòng mồ côi

```
38  @BatchCode VARCHAR(40), @MedicineID INT, @SupplierID INT = NULL, @ReceivedDate DATE, @ExpiryDate
    ↪ DATE,
39  @Quantity INT, @CostPrice DECIMAL(18,2), @SellingPrice DECIMAL(18,2), @BatchID INT OUTPUT
40 AS
41 BEGIN
42  SET NOCOUNT ON;
43  IF @Quantity <= 0 THROW 50010, N'Số lượng nhập phải lớn hơn 0.', 1;
44  IF @ExpiryDate <= CAST(GETDATE() AS date) THROW 50011, N'Hạn sử dụng phải sau ngày hôm nay.', 1;
45  IF @SellingPrice <= 0 THROW 50013, N'Giá bán phải lớn hơn 0.', 1;
46
47  DECLARE @oldMed INT, @oldExp DATE;
48  SELECT @BatchID = BatchID, @oldMed = MedicineID, @oldExp = ExpiryDate
49  FROM dbo.Batches WITH (UPDLOCK, HOLDLOCK) WHERE BatchCode = @BatchCode;
50
51  IF @BatchID IS NOT NULL
52  BEGIN
53   IF @oldMed <> @MedicineID THROW 50014, N'Mã lô này đã thuộc về thuốc khác.', 1;
54   IF @oldExp <> @ExpiryDate THROW 50012, N'Mã lô đã tồn tại với hạn dùng khác. Hãy kiểm tra lại mã
     ↪ lô hoặc hạn dùng.', 1;
55   -- Giá bán của lô GIỮ NGUYÊN (không đổi giá hàng đã có trên kệ); giá nhập = bình quân gia quyền
     ↪ theo số lượng nhập.
56   -- Trong UPDATE, vế phải luôn dùng giá trị CŨ của dòng nên QuantityReceived ở công thức giá là số
     ↪ lượng trước khi cộng.
57   UPDATE dbo.Batches
58      SET CostPrice = CAST((CostPrice * QuantityReceived + @CostPrice * @Quantity) /
        ↪ (QuantityReceived + @Quantity) AS DECIMAL(18,2)),
59          QuantityReceived = QuantityReceived + @Quantity, QuantityRemaining = QuantityRemaining +
            ↪ @Quantity,
60          SupplierID = ISNULL(@SupplierID, SupplierID)
61    WHERE BatchID = @BatchID;
62  END
63  ELSE
64  BEGIN
65   INSERT dbo.Batches(BatchCode,MedicineID,SupplierID,QuantityReceived,QuantityRemaining,CostPrice,
     ↪ SellingPrice,ReceivedDate,ExpiryDate)
66   VALUES(@BatchCode,@MedicineID,@SupplierID,@Quantity,@Quantity,@CostPrice,@SellingPrice,@Received
     ↪ Date,@ExpiryDate);
67   SET @BatchID = SCOPE_IDENTITY();
```

```
68      END
69  END
70  GO
```

Các điểm cần chú ý trong `sp_Lo_Nhap`:

- Kiểm tra đầu vào bằng `THROW` với mã lỗi riêng (50010–50014) để ứng dụng phân loại lỗi: số lượng, hạn dùng, giá bán, lô thuộc thuốc khác, lô đã tồn tại với hạn khác.
- Đọc lô hiện có với gợi ý `UPDLOCK, HOLDLOCK` để hai phiên nhập cùng mã lô không tạo trùng.
- Khi cộng dồn, *giá bán của lô được giữ nguyên* (không đổi giá hàng đã bày bán); giá nhập được tính bình quân gia quyền theo số lượng.

4.5 Dữ liệu mẫu

Script `02_InsertSampleData.sql` tạo hai tài khoản demo, 5 nhóm thuốc, 3 nhà cung cấp, 10 thuốc và các lô có *ngày tính theo ngày chạy script*, nhờ đó luôn có đủ tình huống để kiểm thử (Bảng 4.10); hai hóa đơn mẫu phục vụ kiểm thử tra cứu và báo cáo, cùng 11 phiếu nhập mẫu (mỗi lô một phiếu, riêng phiếu `PN000005` có hai dòng) phục vụ kiểm thử lịch sử nhập kho.

Bảng 4.10: Các tình huống có sẵn trong dữ liệu mẫu

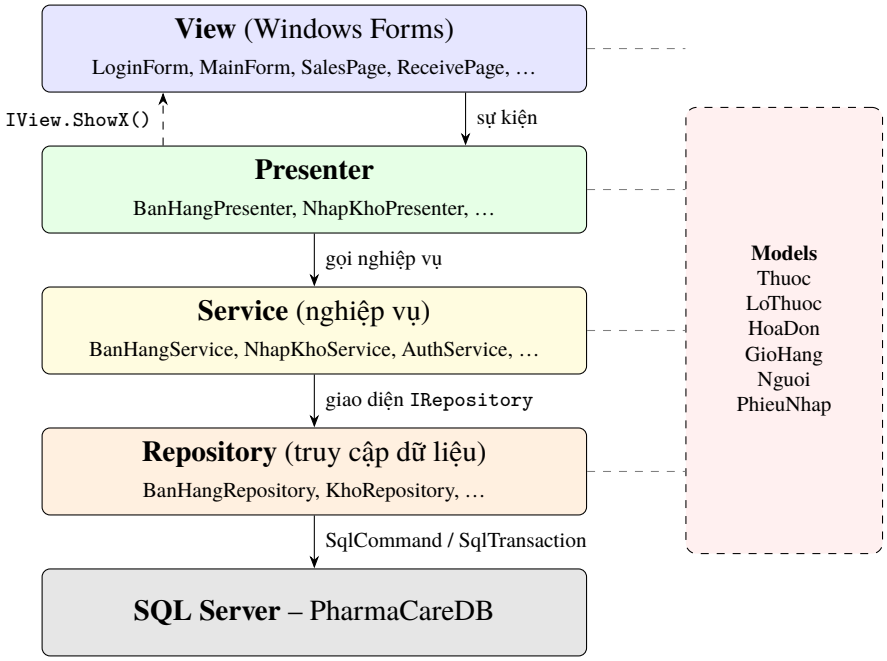
Tình huống	**Lô / dữ liệu**
Lô đã hết hạn	`LOT-PA-000` (Paracetamol), `LOT-AM-000` (Amoxicillin)
Lô sắp hết hạn ($\leq$ 30 ngày)	`LOT-OR-001` (Oresol, còn 18 ngày), `LOT-VT-001` (Vitamin tổng hợp, còn 15 ngày)
Lô sắp hết hàng	`LOT-MEN-001` (Men tiêu hóa, tồn 8)
Thuốc kê đơn	Amoxicillin 500mg (`MED010`)
Tài khoản	`duocsi` (Dược sĩ), `quanly` (Quản lý kho)
Hóa đơn mẫu	Hai hóa đơn: 18 000 đ (khách lẻ) và 28 000 đ (Nguyễn Văn A)
Phiếu nhập mẫu	11 phiếu, tổng giá trị 5 598 000 đ; riêng nhà cung cấp Công ty Dược An Khang có 5 phiếu, tổng 1 418 000 đ

Chương 5

Kiến trúc chương trình

5.1 Tổng quan kiến trúc

PharmaCare được tổ chức theo mô hình **MVP** kết hợp **nhiều lớp** [7, 8]. Giao diện (View) chỉ hiển thị và phát sinh sự kiện; Presenter nhận sự kiện, chuyển đổi dữ liệu thô từ giao diện, gọi tầng Service và yêu cầu View hiển thị kết quả; Service chứa toàn bộ quy tắc nghiệp vụ (FEFO, kiểm tra hạn dùng, kiểm tra tồn, phân quyền); Repository là nơi duy nhất làm việc với SQL Server. Các lớp Model (thực thể) được dùng chung giữa các lớp.



Hình 5.1: Kiến trúc MVP kết hợp nhiều lớp của PharmaCare

5.2 Tổ chức thư mục mã nguồn

Danh sách mã 5.1: Cấu trúc thư mục dự án

```
PharmaCare/
|- Models/        Thực thể: Nguoi -> DuocSi / QuanLyKho, Thuoc, LoThuoc, HoaDon, GioHang, PhieuNhap
|- Exceptions/    ThuocHetHanException, KhongDuTonKhoException, MaTrungException (cùng cha
↪  NghiepVuException)
|- Views/         IViews.cs (interface từng màn hình) + LoginForm, MainForm (MDI), SalesPage,
↪  ReceivePage, ...
|- Presenters/    Điều phối: TryParse an toàn, bắt ngoại lệ, yêu cầu View hiển thị
|- Services/      Nghiệp vụ: FEFO, kiểm tra tồn, hạn dùng, đăng nhập, phân quyền
|- Repositories/  Truy cập dữ liệu: SqlConnection / SqlTransaction / SqlParameter, gọi Stored
↪  Procedure
|- Printing/      InvoiceRenderer: vẽ hóa đơn dùng chung cho Preview và in thật
|- Helpers/       PasswordHasher, ImageHelper, AppEvents, AppSession
|- Ui.cs, GridSorter.cs, Db.cs, DbSetup.cs, AppConfig.cs
Database/         01_CreateDatabase.sql, 02_InsertSampleData.sql, 03_StoredProcedures.sql,
↪  04_BackupDatabase.sql
Docs/             TestCases.md, báo cáo kiểm thử, thuyết minh
```

5.3 Trách nhiệm từng lớp

Bảng 5.1: Trách nhiệm và ví dụ của từng lớp

Lớp	Trách nhiệm	Ví dụ trong đề tài
View	Hiển thị, nhận thao tác, trả về *chuỗi thuần*; không chứa SQL hay luật nghiệp vụ	`SalesPage` cài đặt `IBanHangView`
Presenter	`TryParse`, gọi Service, bắt `NghiepVuException`, yêu cầu View hiện lỗi/kết quả	`BanHangPresenter.ThanhToan`
Service	Quy tắc nghiệp vụ, phân quyền, điều phối nhiều Repository; không dùng `MessageBox`	`BanHangService.PhanBoFefo`
Repository	Duy nhất làm việc với SQL Server; truy vấn tham số hóa; quản lý giao dịch	`BanHangRepository.LuuHoaDon`
Model	Thực thể và quy tắc bất biến của chính nó	`Nguoi`, `LoThuoc`, `GioHang`

Nhờ View chỉ làm việc với *giao diện* (`IBanHangView`, `INhapKhoView`, …) và Service chỉ làm việc với *giao diện Repository* (`IBanHangRepository`, …), có thể thay một lớp bằng bản giả lập khi kiểm thử; ví dụ `BanHangService` nhận thêm hàm cung cấp ngày hiện tại (`Func<DateTime>`) để kiểm thử logic hạn dùng mà không phụ thuộc đồng hồ máy.

5.4 Thiết kế lớp

5.4.1 Hệ thống phân cấp người dùng

Lớp trừu tượng `Nguoi` đóng gói tên đăng nhập và họ tên (có kiểm tra hợp lệ trong `set`) và khai báo phương thức `CoQuyen(Quyen)`. Hai lớp con `DuocSi` và `QuanLyKho` *ghi đè* tập quyền, thể

hiện tính kế thừa và đa hình (Danh sách mã 5.2).

Danh sách mã 5.2: Lớp `Nguoi` và hai lớp con: kế thừa, đa hình, đóng gói

```
namespace PharmaCare.Models;

public enum Quyen { XemDanhMuc, SuaThuoc, NhapKho, BanHang, XemHoaDon, XemCanhBao, XemBaoCao, DoiKeDon }

/// <summary>Lớp cha của người dùng hệ thống (kế thừa + đa hình + đóng gói).</summary>
public abstract class Nguoi
{
    string _tenDangNhap = "", _hoTen = "";

    protected Nguoi(string tenDangNhap, string hoTen) { TenDangNhap = tenDangNhap; HoTen = hoTen; }

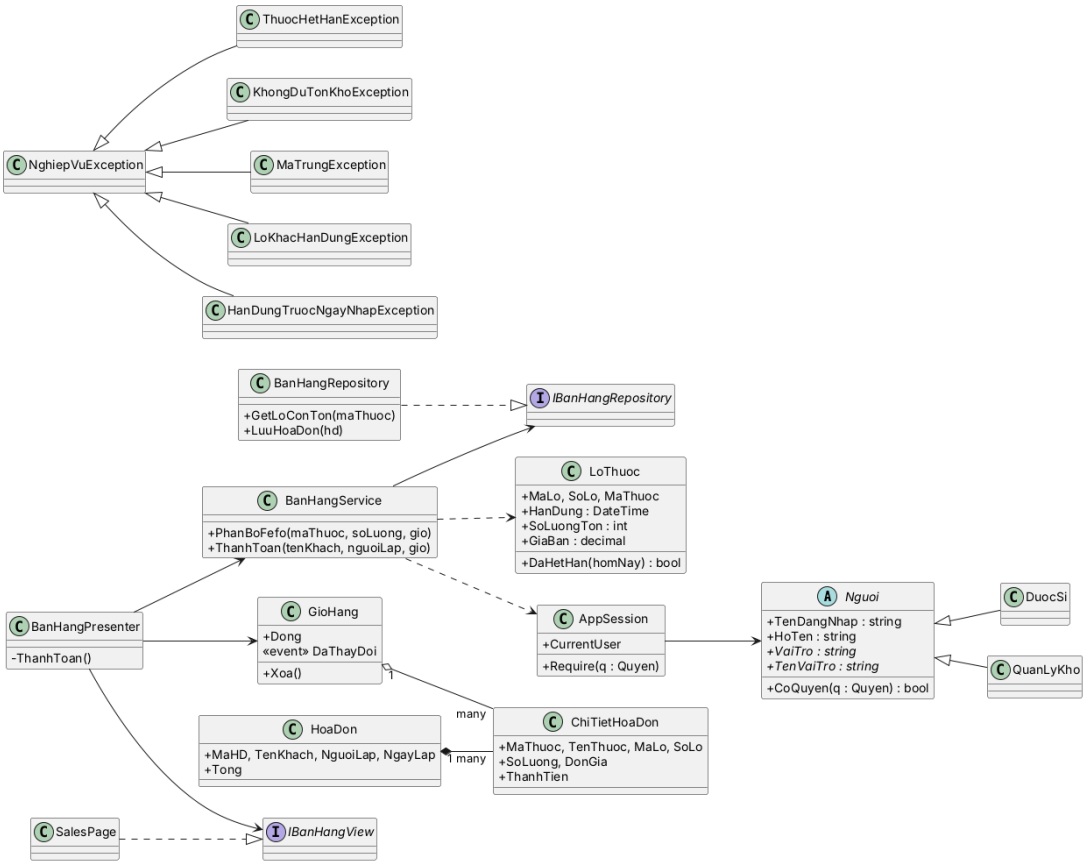
    public string TenDangNhap
    {
        get => _tenDangNhap;
        private set => _tenDangNhap = string.IsNullOrWhiteSpace(value) ? throw new ArgumentException("Tên đăng nhập không được rỗng.") : value.Trim();
    }
    public string HoTen
    {
        get => _hoTen;
        private set => _hoTen = string.IsNullOrWhiteSpace(value) ? throw new ArgumentException("Họ tên không được rỗng.") : value.Trim();
    }

    public abstract string VaiTro { get; }
    public abstract string TenVaiTro { get; }
    protected abstract IReadOnlySet<Quyen> QuyenDuocCap { get; }
    public bool CoQuyen(Quyen q) => QuyenDuocCap.Contains(q);
}

public sealed class DuocSi : Nguoi
{
    static readonly HashSet<Quyen> _q = new() { Quyen.XemDanhMuc, Quyen.BanHang, Quyen.XemHoaDon, Quyen.XemCanhBao, Quyen.DoiKeDon };
    public DuocSi(string tenDangNhap, string hoTen) : base(tenDangNhap, hoTen) { }
    public override string VaiTro => "DUOCSI";
    public override string TenVaiTro => "Dược sĩ bán hàng";
    protected override IReadOnlySet<Quyen> QuyenDuocCap => _q;
}

public sealed class QuanLyKho : Nguoi
{
    static readonly HashSet<Quyen> _q = new() { Quyen.XemDanhMuc, Quyen.SuaThuoc, Quyen.NhapKho, Quyen.XemHoaDon, Quyen.XemCanhBao, Quyen.XemBaoCao, Quyen.DoiKeDon };
    public QuanLyKho(string tenDangNhap, string hoTen) : base(tenDangNhap, hoTen) { }
    public override string VaiTro => "QUANLYKHO";
    public override string TenVaiTro => "Quản lý kho";
    protected override IReadOnlySet<Quyen> QuyenDuocCap => _q;
}

public sealed record TaiKhoanDto(string TenDangNhap, string MatKhauHash, string HoTen, string VaiTro, bool HoatDong);
```



Hình 5.2: Sơ đồ lớp (Class Diagram) – Models, Services và Repositories chính

5.4.2 Hệ thống ngoại lệ nghiệp vụ

Mọi lỗi do vi phạm quy tắc nghiệp vụ đều là `NghiepVuException` hoặc lớp con của nó, nhờ đó Presenter bắt riêng loại này và báo lỗi bằng tiếng Việt dễ hiểu; lỗi hệ thống khác (mất kết nối…) được báo chung.

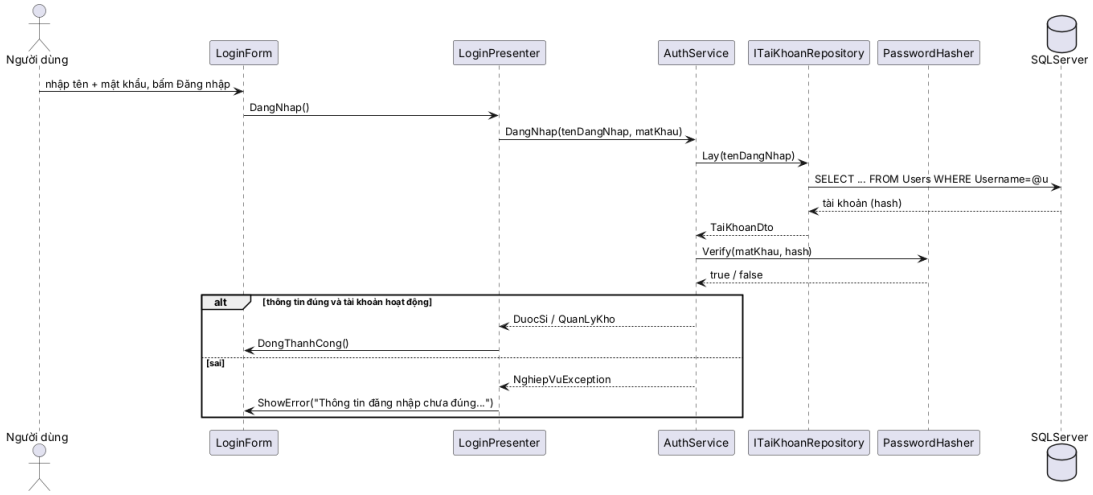
Bảng 5.2: Các ngoại lệ nghiệp vụ

Ngoại lệ	Khi nào xảy ra
`ThuocHetHanException`	Bán hoặc nhập thuốc có hạn dùng là hôm nay hoặc đã qua
`KhongDuTonKhoException`	Số lượng bán vượt tồn còn bán được; kho đổi khi đang thanh toán
`MaTrungException`	Trùng mã thuốc hoặc mã lô
`LoKhacHanDungException`	Nhập lại mã lô đã có nhưng hạn dùng khác hạn hiện có
`HanDungTruocNgayNhapException`	Hạn dùng nhỏ hơn ngày nhập

5.5 Luồng xử lý chính

5.5.1 Luồng đăng nhập

`LoginForm` gửi tên đăng nhập và mật khẩu cho `LoginPresenter`; Presenter gọi `AuthService.DangNhap`, Service lấy tài khoản qua `ITaiKhoanRepository`, kiểm tra trạng thái hoạt động và so khớp mật khẩu bằng `PasswordHasher.Verify`. Sai tên hay sai mật khẩu đều trả *cùng một thông báo* để không lộ tài khoản nào tồn tại. Thành công thì tạo đối tượng `DuocSi` hoặc `QuanLyKho`, lưu vào `AppSession` và mở `MainForm`.



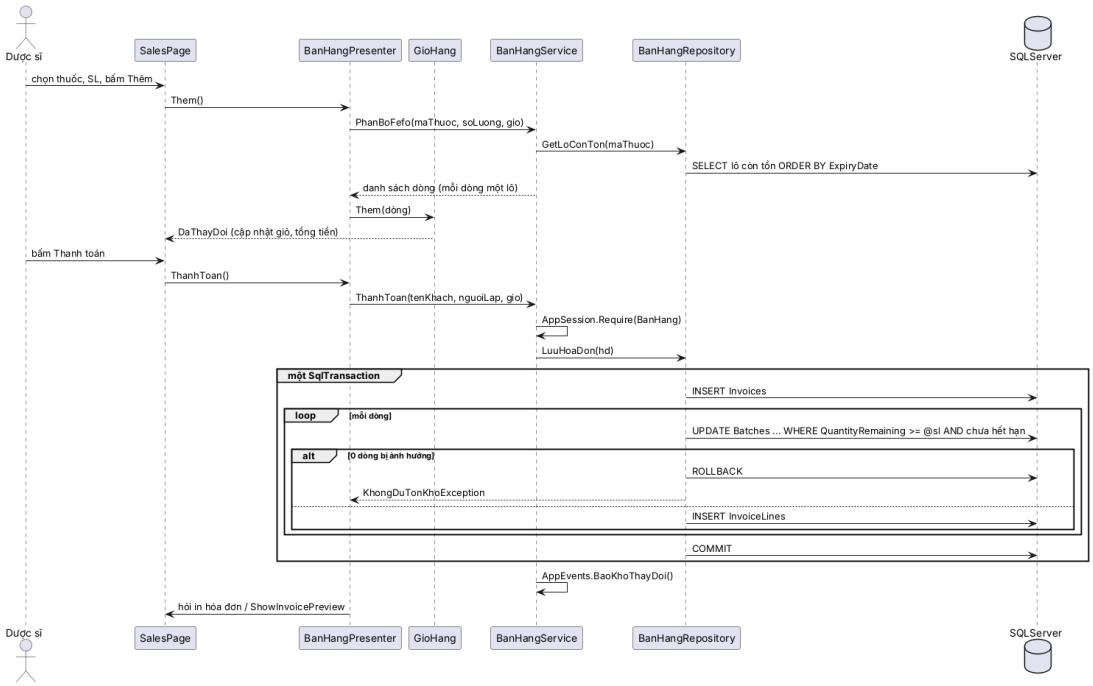
Hình 5.3: Sơ đồ tuần tự: đăng nhập

5.5.2 Luồng bán hàng theo FEFO

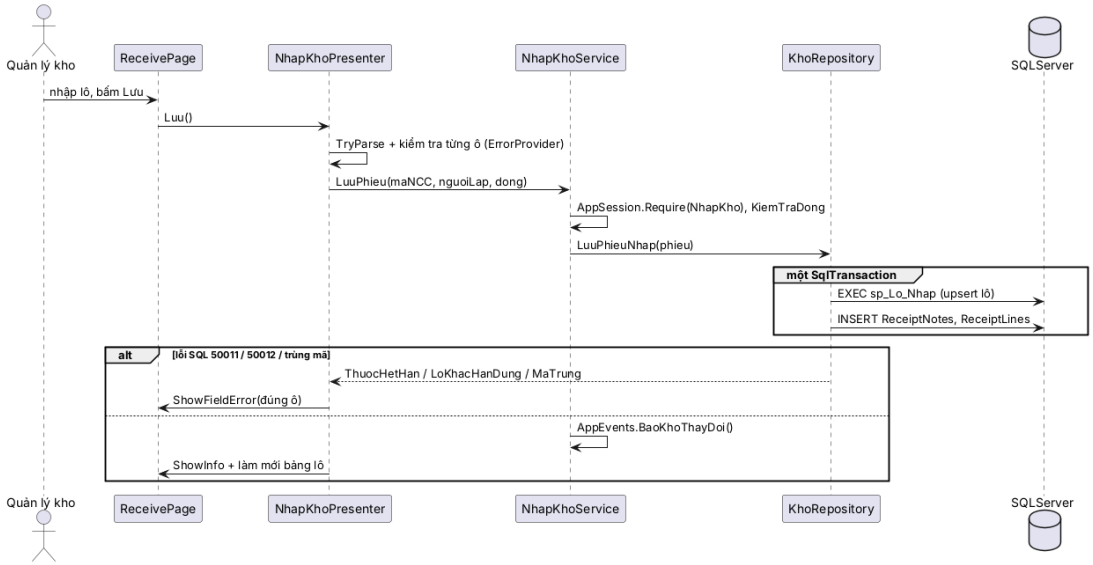
Khi người bán bấm *Thêm*, `BanHangPresenter` gọi `BanHangService.PhanBoFefo`, hàm này trả về danh sách dòng (mỗi dòng một lô) và Presenter thêm vào giỏ hàng. Khi bấm *Thanh toán*, Service kiểm tra quyền, tạo `HoaDon` và gọi `BanHangRepository.LuuHoaDon` để ghi hóa đơn và trừ kho trong một giao dịch; thành công thì phát sự kiện `AppEvents.KhoThayDoi` để trang Cảnh báo tự cập nhật. Chi tiết thuật toán xem mục 6.3.

5.5.3 Luồng nhập kho

`ReceivePage` → `NhapKhoPresenter` (TryParse, kiểm tra từng ô, gắn lỗi vào `ErrorProvider`) → `NhapKhoService` (kiểm tra quyền, hạn dùng, giá) → `KhoRepository.LuuPhieuNhap` (một giao dịch: `sp_Lo_Nhap`, ghi `ReceiptNotes` và `ReceiptLines`). Trang lịch sử nhập dùng cùng mẫu với trang Hóa đơn: `ReceiptsPage` → `PhieuNhapPresenter` → `NhapKhoService.GetPhieuNhap` (kiểm tra quyền `NhapKho` và khoảng ngày) → `KhoRepository.GetPhieuNhap`.



Hình 5.4: Sơ đồ tuần tự: bán hàng và thanh toán



Hình 5.5: Sơ đồ tuần tự: nhập kho theo lô

5.6 Phân quyền hai tầng

Quyền được kiểm tra ở hai nơi: (1) *giao diện* ẩn hoặc khóa chức năng không đủ quyền (sidebar, nút Thêm/Sửa/Xóa); (2) *tầng Service* gọi `AppSession.Require(Quyen.…)` trước các thao tác ghi dữ liệu hoặc xem dữ liệu nhạy cảm. Nhờ lớp thứ hai, dù một màn hình vô tình mở lộ chức năng, thao tác vẫn bị từ chối (Danh sách mã 5.3).

Danh sách mã 5.3: `AppSession`: người đang đăng nhập và kiểm tra quyền ở Service

```
using PharmaCare.Exceptions;
using PharmaCare.Models;
namespace PharmaCare.Helpers;

/// <summary>Người dùng đang đăng nhập. Service gọi Require() để chặn thao tác không đủ quyền ngay cả khi View bị bỏ qua
/// (trước đây quyền chỉ được kiểm tra bằng cách ẩn/khóa nút ở View).</summary>
public static class AppSession
{
    public static Nguoi? CurrentUser { get; set; }

    public static void Require(Quyen q)
    {
        var u = CurrentUser ?? throw new NghiepVuException("Chưa đăng nhập.");
        if (!u.CoQuyen(q))
            throw new NghiepVuException($"Tài khoản "{u.TenDangNhap}" ({u.TenVaiTro}) không có quyền thực hiện thao tác này.");
    }
}
```

5.7 Cơ chế sự kiện làm mới dữ liệu

Khi kho thay đổi (bán hoặc nhập thành công), Service gọi `AppEvents.BaoKhoThayDoi()`; các trang đang mở như `AlertsPage` đăng ký sự kiện `KhoThayDoi` và tự tải lại, nên người dùng không phải đóng và mở lại ứng dụng. Giỏ hàng `GioHang` cũng phát sự kiện `DaThayDoi` để View cập nhật danh sách và tổng tiền.

Chương 6

Mô tả các chức năng chính

Chương này trình bày cách cài đặt những phần có nhiều quyết định kỹ thuật nhất: khởi tạo cơ sở dữ liệu, bảo mật mật khẩu, thuật toán FEFO, giao dịch bán hàng, nhập kho, cảnh báo hạn dùng và in hóa đơn. Các đoạn mã được trích trực tiếp từ kho mã nguồn; mã đầy đủ xem Phụ lục C.

6.1 Khởi tạo và tự cấu hình cơ sở dữ liệu

Để người chấm có thể chạy ứng dụng mà không phải cấu hình thủ công, lớp `DbSetup` thực hiện khi mở ứng dụng:

1. Dò lần lượt các máy chủ SQL Server thường gặp (`.\ SQLEXPRESS`, `(localdb)\ MSSQLLocalDB`, `.`, tên máy) với thời gian chờ 8 giây (20 giây cho LocalDB vì lần khởi động đầu tiên chậm).
2. Kiểm tra trạng thái cơ sở dữ liệu bằng `CheckState`: *chưa có* hoặc *bản cũ thiếu bảng Users* thì chạy cả ba script nhúng sẵn; *đủ bảng nhưng thiếu thủ tục lưu sẵn* thì *chỉ tạo lại thủ tục* (`CREATE OR ALTER`), giữ nguyên dữ liệu; đã đủ thì không làm gì.
3. Nếu thất bại, màn hình đăng nhập hiện nút *Chi tiết lỗi* cho biết nguyên nhân từng máy chủ đã thử.

Điểm quan trọng là việc thiếu một thủ tục lưu sẵn *không còn làm mất dữ liệu*: trước đây điều kiện kiểm tra sai có thể khiến script tạo lại toàn bộ bảng.

Danh sách mã 6.1: Kiểm tra trạng thái CSDL để quyết định tạo mới hay chỉ tạo lại thủ tục (`DbSetup.cs`)

```
63      enum DbState { CanTaoMoi, ThieuProcedure, SanSang }
64
65      /// <summary>Chưa có database hoặc thiếu bảng Users (bản cũ) => tạo lại; có đủ bảng nhưng thiếu
        ↪  sp_Lo_Nhap => chỉ tạo lại stored procedure (không đụng dữ liệu).</summary>
66      static DbState CheckState(SqlConnection cn)
67      {
68          using var cmd = new SqlCommand(
69              "SELECT CASE WHEN DB_ID(@db) IS NULL THEN 0 WHEN OBJECT_ID(@tbl) IS NULL THEN 0 WHEN
                ↪  OBJECT_ID(@sp) IS NULL THEN 1 ELSE 2 END", cn);
```

```
        cmd.Parameters.AddWithValue("@db", AppConfig.Database);
        cmd.Parameters.AddWithValue("@tbl", AppConfig.Database + ".dbo.Users");
        cmd.Parameters.AddWithValue("@sp", AppConfig.Database + ".dbo.sp_Lo_Nhap");
        return Convert.ToInt32(cmd.ExecuteScalar()) switch { 0 => DbState.CanTaoMoi, 1 =>
        ↪  DbState.ThieuProcedure, _ => DbState.SanSang };
    }

    // Script nhúng sẵn trong ứng dụng, chạy lần lượt: tạo CSDL -> dữ liệu mẫu -> stored procedure
    static readonly string[] Scripts = { "01_CreateDatabase.sql", "02_InsertSampleData.sql",
    ↪  "03_StoredProcedures.sql" };
```

6.2 Bảo mật mật khẩu

Mật khẩu được băm bằng PBKDF2-SHA256 với salt ngẫu nhiên 16 byte và 100 000 vòng lặp; chuỗi lưu có dạng `PBKDF2$vòng$salt$hash`, nên có thể tăng số vòng lặp trong tương lai mà vẫn xác thực được mật khẩu cũ. Hàm `Verify` so sánh bằng `FixedTimeEquals` để thời gian xử lý không phụ thuộc vào vị trí byte khác biệt.

Danh sách mã 6.2: Băm và xác thực mật khẩu bằng PBKDF2-SHA256

```
using System.Security.Cryptography;
namespace PharmaCare.Helpers;

/// <summary>PBKDF2-SHA256. Định dạng lưu: PBKDF2$iterations$saltBase64$hashBase64</summary>
public static class PasswordHasher
{
    const int Iterations = 100_000, SaltSize = 16, HashSize = 32;

    public static string Hash(string password)
    {
        byte[] salt = RandomNumberGenerator.GetBytes(SaltSize);
        byte[] hash = Rfc2898DeriveBytes.Pbkdf2(password, salt, Iterations,
        ↪  HashAlgorithmName.SHA256, HashSize);
        return
        ↪  $"PBKDF2${Iterations}${Convert.ToBase64String(salt)}${Convert.ToBase64String(hash)}";
    }

    public static bool Verify(string password, string stored)
    {
        var p = stored.Split('$');
        if (p.Length != 4 || p[0] != "PBKDF2" || !int.TryParse(p[1], out int it)) return false;
        try
        {
            byte[] salt = Convert.FromBase64String(p[2]);
            byte[] expected = Convert.FromBase64String(p[3]);
            byte[] actual = Rfc2898DeriveBytes.Pbkdf2(password, salt, it, HashAlgorithmName.SHA256,
            ↪  expected.Length);
            return CryptographicOperations.FixedTimeEquals(actual, expected);
        }
        catch (FormatException) { return false; }
    }
}
```

6.3 Thuật toán bán theo FEFO

Hàm `PhanBoFefo` nhận mã thuốc, số lượng cần bán và giỏ hàng hiện tại, trả về danh sách dòng bán (mỗi dòng một lô). Các bước:

1. Kiểm tra số lượng > 0 và thuốc tồn tại.
2. Gom số lượng *đã nằm trong giỏ* theo từng lô (`daGiu`), để thêm cùng một thuốc nhiều lần không bán vượt tồn.
3. Lấy các lô còn tồn, *loại lô đã hết hạn*, sắp xếp theo hạn dùng tăng dần (hòa thì theo mã lô).
4. Nếu không còn lô bán được: báo `ThuocHetHanException` (chỉ còn lô hết hạn) hoặc `KhongDuTonKhoException` (hết hàng).
5. Duyệt từng lô: số lượng trống = tồn − đã giữ; lấy min(trống, còn cần) rồi chuyển sang lô kế tiếp cho đến khi đủ.
6. Nếu vẫn thiếu, báo không đủ tồn kèm *số lượng tối đa còn bán được*.

Danh sách mã 6.3: `BanHangService.PhanBoFefo`: phân bổ theo hạn dùng gần nhất

```
21      /// <summary>
22      /// FEFO: bán lô còn hạn có hạn dùng gần nhất trước, tràn sang lô kế tiếp nếu chưa đủ.
23      /// Tính cả phần đã giữ chỗ trong giỏ để thêm cùng một thuốc nhiều lần không bán vượt tồn.
24      /// </summary>
25      public IReadOnlyList<ChiTietHoaDon> PhanBoFefo(int maThuoc, int soLuong,
        ↪  IEnumerable<ChiTietHoaDon> gio)
26      {
27          if (soLuong <= 0) throw new NghiepVuException("Số lượng phải lớn hơn 0.");
28          string ten = _repo.LayTenThuoc(maThuoc) ?? throw new NghiepVuException("Không tìm thấy
            ↪  thuốc đã chọn.");
29
30          var daGiu = gio.GroupBy(c => c.MaLo).ToDictionary(g => g.Key, g => g.Sum(c => c.SoLuong));
31          var cacLo = _repo.GetLoConTon(maThuoc);
32          var homNay = _homNay();
33          var loBanDuoc = cacLo.Where(l => !l.DaHetHan(homNay)).OrderBy(l => l.HanDung).ThenBy(l =>
            ↪  l.MaLo).ToList();
34
35          if (loBanDuoc.Count == 0)
36          {
37              if (cacLo.Count > 0) throw new ThuocHetHanException(ten);          // chỉ còn lô hết
                ↪  hạn
38              throw new KhongDuTonKhoException(ten, 0);
39          }
40
41          var ketQua = new List<ChiTietHoaDon>();
42          int can = soLuong, tongConLai = 0;
43          foreach (var lo in loBanDuoc)
44          {
45              int trong = lo.SoLuongTon - daGiu.GetValueOrDefault(lo.MaLo);
46              if (trong <= 0) continue;
47              tongConLai += trong;
```

```
48              if (can == 0) continue;
49              int lay = Math.Min(trong, can);
50              ketQua.Add(new ChiTietHoaDon(maThuoc, ten, lo.MaLo, lo.SoLo, lay, lo.GiaBan));
51              can -= lay;
52          }
53          if (can > 0) throw new KhongDuTonKhoException(ten, tongConLai);
54          return ketQua;
55      }
```

Ví dụ. Paracetamol có lô `LOT-PA-000` (đã hết hạn, bỏ qua), `LOT-PA-001` (hạn xa) và lô mới `LO-X` (tồn 10, hạn gần hơn). Bán 30 viên: hệ thống lấy 10 từ `LO-X` rồi 20 từ `LOT-PA-001`, giỏ hàng có hai dòng. Nếu người bán thêm tiếp 5 viên khi `LOT-PA-001` còn đúng 380 trừ 20 đã giữ, phần giữ chỗ được tính nên không bao giờ vượt tồn.

6.4 Giao dịch bán hàng và trừ kho

Việc ghi hóa đơn và trừ kho phải thành công hoặc thất bại *cùng nhau*. `LuuHoaDon` mở một `SqlConnection` và một `SqlTransaction` dùng chung cho mọi lệnh: thêm hóa đơn, rồi với mỗi dòng thực hiện *trừ kho có điều kiện* và thêm dòng hóa đơn. Câu lệnh trừ kho

```
UPDATE Batches SET QuantityRemaining = QuantityRemaining - @sl
WHERE BatchID = @ma AND QuantityRemaining >= @sl
  AND ExpiryDate >= CAST(GETDATE() AS date)
```

vừa kiểm tra đủ tồn, vừa kiểm tra chưa hết hạn *ngay trong cùng một lệnh nguyên tử*. Nếu số dòng bị ảnh hưởng bằng 0 (máy khác đã bán trước, hoặc lô vừa hết hạn) thì ném `KhongDuTonKhoException` và toàn bộ giao dịch bị `Rollback()`; không có hóa đơn mồ côi, tồn kho không bị âm.

Danh sách mã 6.4: `BanHangRepository.LuuHoaDon`: một giao dịch cho hóa đơn và trừ kho

```
25      /// <summary>Bán thuốc trong MỘT transaction: Invoices -> mỗi dòng: trừ kho có điều kiện ->
        ↪  InvoiceLines.
26      /// Số dòng bị ảnh hưởng = 0 hoặc lỗi bất kỳ => Rollback toàn bộ.</summary>
27      public int LuuHoaDon(HoaDon hd)
28      {
29          using var cn = Db.OpenConnection();
30          using var tx = cn.BeginTransaction();
31          try
32          {
33              int maHD;
34              using (var cmd = new SqlCommand("INSERT
                ↪  dbo.Invoices(InvoiceDate,CustomerName,TotalAmount,CreatedBy) OUTPUT
                ↪  INSERTED.InvoiceID VALUES(GETDATE(),@name,@total,@user)", cn, tx))
35              {
36                  cmd.Parameters.Add(Db.P("@name", hd.TenKhach));
37                  cmd.Parameters.Add(Db.M("@total", hd.Tong));
38                  cmd.Parameters.Add(Db.P("@user", hd.NguoiLap));
```

```
39                maHD = Convert.ToInt32(cmd.ExecuteScalar());
40            }
41            foreach (var c in hd.ChiTiet)
42            {
43                using var tru = new SqlCommand(
44                    "UPDATE dbo.Batches SET QuantityRemaining=QuantityRemaining-@sl WHERE
                      ↪  BatchID=@ma AND QuantityRemaining>=@sl AND ExpiryDate>=CAST(GETDATE() AS
                      ↪  date)", cn, tx);
45                tru.Parameters.Add(Db.P("@sl", c.SoLuong));
46                tru.Parameters.Add(Db.P("@ma", c.MaLo));
47                if (tru.ExecuteNonQuery() == 0) throw new KhongDuTonKhoException(c.TenThuoc, 0);
48
49                using var ct = new SqlCommand("INSERT
                  ↪  dbo.InvoiceLines(InvoiceID,BatchID,Quantity,UnitPrice)
                  ↪  VALUES(@hd,@lo,@sl,@gia)", cn, tx);
50                ct.Parameters.Add(Db.P("@hd", maHD));
51                ct.Parameters.Add(Db.P("@lo", c.MaLo));
52                ct.Parameters.Add(Db.P("@sl", c.SoLuong));
53                ct.Parameters.Add(Db.M("@gia", c.DonGia));
54                ct.ExecuteNonQuery();
55            }
56            tx.Commit();
57            return maHD;
58        }
59        catch { try { tx.Rollback(); } catch { /* kết nối đã đứt */ } throw; }
60    }
```

6.4.1 Xử lý kết quả ở Presenter

Sau khi lưu, Presenter phân biệt rõ hai giai đoạn (Danh sách mã 6.5):

- *Giai đoạn lưu*: lỗi tồn kho ⇒ báo đã rollback, **làm trống giỏ** và nạp lại danh sách thuốc (vì các dòng trong giỏ trỏ vào lô cũ không còn đúng); lỗi nghiệp vụ khác ⇒ chỉ hiện thông báo; lỗi hệ thống ⇒ báo thất bại, đã rollback.
- *Giai đoạn in*: hóa đơn *đã được lưu*. Nếu xem trước hoặc in lỗi (ví dụ máy không có máy in) thì thông báo rõ "hóa đơn đã lưu, không cần thanh toán lại", tránh để người dùng bán lại lần hai.

Danh sách mã 6.5: `BanHangPresenter.ThanhToan`: tách bước lưu và bước in

```
34    void ThanhToan()
35    {
36        // Bước 1: LƯU hóa đơn (transaction). Chỉ lỗi ở bước này mới được báo là "đã rollback".
37        HoaDon hd;
38        try { hd = _service.ThanhToan(_view.TenKhach, _nguoiLap, _gio.Dong.ToList()); }
39        catch (KhongDuTonKhoException ex)
40        {
41            // Tồn kho đã đổi (máy khác bán trước...): các dòng trong giỏ trỏ vào lô cũ nên không
                  ↪  còn đúng -> làm trống giỏ + nạp lại danh sách.
42            _gio.Xoa(); Start();
43            _view.ShowError("Thanh toán thất bại; dữ liệu đã được rollback, chưa có hóa đơn nào
                  ↪  được lưu.\n" + ex.Message +
```

```
                            "\n\nGiỏ hàng đã được làm trống vì tồn kho đã thay đổi. Vui lòng thêm
                            ↪ lại thuốc.");
            return;
        }
        catch (NghiepVuException ex) { _view.ShowError(ex.Message); return; }      // ví dụ: giỏ
        ↪ trống, không đủ quyền
        catch (Exception ex) { _view.ShowError("Thanh toán thất bại; dữ liệu đã được rollback.\n" +
        ↪ ThongBao(ex)); return; }

        // Bước 2: hóa đơn ĐÃ commit. Lỗi ở bước in/xem trước KHÔNG được báo là thất bại (tránh
        ↪ người dùng bán lại lần hai).
        _gio.Xoa();
        try
        {
            if (_view.HoiInHoaDon($"Thanh toán thành công!\nMã hóa đơn: HD{hd.MaHD:000000}\nTổng
            ↪ tiền: {hd.Tong:N0} ₫\n\nBạn có muốn xem và in hóa đơn không?"))
                _view.ShowInvoicePreview(hd);
        }
        catch (Exception ex)
        {
            _view.ShowError($"Hóa đơn HD{hd.MaHD:000000} ĐÃ được lưu thành công (không cần thanh
            ↪ toán lại), nhưng không mở được bản xem trước / in hóa đơn.\n\nChi tiết:
            ↪ {ex.Message}\nKiểm tra máy in mặc định rồi in lại từ trang Hóa đơn.");
        }
        Start();
    }
}
```

6.5 Nhập kho theo lô

Nhập kho dùng thủ tục `sp_Lo_Nhap` (Danh sách mã 4.1) trong một giao dịch cùng việc ghi phiếu nhập. Presenter kiểm tra từng ô và gắn lỗi vào đúng ô nhờ `ErrorProvider`: hạn dùng là hôm nay hoặc quá khứ ⇒ ô Hạn sử dụng; hạn dùng trước ngày nhập ⇒ cũng ô Hạn sử dụng; ngày nhập không hợp lệ ⇒ ô Ngày nhập (không tự đổi thành hôm nay); mã lô đã có với hạn khác ⇒ thông báo hướng dẫn nhập đúng hạn của lô. Khi chọn một thuốc, ô Giá bán được điền sẵn giá niêm yết nhưng vẫn sửa được vì giá bán thực tế là giá của từng lô. Ở tầng Repository, mã lỗi SQL được ánh xạ sang ngoại lệ nghiệp vụ tương ứng (ví dụ mã 50011 → `ThuocHetHanException`), nên kết quả nhất quán dù hạn dùng được kiểm tra ở C# hay ở SQL Server.

Lịch sử nhập kho do `KhoRepository.GetPhieuNhap` và `GetChiTietPhieuNhap` đảm nhiệm: truy vấn tham số hóa nối `ReceiptNotes` với `Suppliers` (`LEFTJOIN`, phiếu không ghi nhà cung cấp hiện là "(Không rõ)") và nối `ReceiptLines` với `Batches`, `Medicines` để lấy chi tiết. Điều kiện nhà cung cấp viết dạng `(@sup IS NULL OR SupplierID = @sup)` nên một câu lệnh phục vụ cả trường hợp "tất cả nhà cung cấp"; ký tự `%`, `_`, `[` trong ô tìm kiếm được thoát để không bị hiểu là ký tự đại diện của `LIKE`. `NhapKhoService` kiểm tra quyền `NhapKho` và từ chối khoảng ngày sai bằng ngoại lệ nghiệp vụ; Presenter hiển thị ngoại lệ này như một cảnh báo, giữ nguyên bảng đang xem.

6.6 Cảnh báo hạn dùng

Trang Cảnh báo gọi `sp_Lo_CanhBao`, hiển thị bảng các lô cần chú ý và tô màu theo trạng thái: đỏ (đã hết hạn), vàng (sắp hết hạn, trong 30 ngày), cam (sắp hết hàng, tồn ≤ 10). Có thể lọc theo trạng thái ngay trên dữ liệu đã tải bằng `BindingSource.Filter`. Trang đăng ký sự kiện `AppEvents.KhoThayDoi` nên tự tải lại sau mỗi lần bán hoặc nhập kho thành công.

6.7 In hóa đơn nhiều trang

Hóa đơn được vẽ bằng một hàm duy nhất `InvoiceRenderer.DrawInvoice`, dùng chung cho Print Preview và in thật. Hàm nhận tham số `ref int next` là chỉ số dòng chi tiết đầu tiên cần vẽ: mỗi trang vẽ tiêu đề, đầu bảng, rồi các dòng cho đến khi còn chừa chỗ cho phần tổng cộng; nếu còn dòng chưa in thì trả về `true` để `PrintPageEventArgs.HasMorePages` yêu cầu in trang tiếp theo, và phần *TỔNG CỘNG* chỉ được in ở trang cuối. Mỗi trang in ít nhất một dòng để không bao giờ lặp vô hạn. Sự kiện `BeginPrint` đặt lại vị trí in, nên mở Preview nhiều lần vẫn bắt đầu từ trang đầu.

Danh sách mã 6.6: Vòng lặp phân trang trong `InvoiceRenderer.DrawInvoice`

```
56          int drawn = 0;
57          while (next < hd.ChiTiet.Count)
58          {
59              // luôn chừa chỗ cho phần tổng cộng; mỗi trang in ít nhất 1 dòng để không bao giờ lặp
                ↪ vô hạn
60              if (drawn > 0 && y + RowH > bounds.Bottom - FooterH) break;
61              var c = hd.ChiTiet[next];
62              g.DrawString(c.TenThuoc, fText, Brushes.Black, new RectangleF(x[0], y, x[1] - x[0] - 4,
                ↪ RowH), ellipsis);
63              g.DrawString(c.SoLo, fText, Brushes.Black, new RectangleF(x[1], y, x[2] - x[1] - 4,
                ↪ RowH), ellipsis);
64              g.DrawString(c.SoLuong.ToString(), fText, Brushes.Black, new RectangleF(x[2], y, x[3] -
                ↪ x[2] - 4, RowH), right);
65              g.DrawString(c.DonGia.ToString("N0"), fText, Brushes.Black, new RectangleF(x[3], y,
                ↪ x[4] - x[3] - 4, RowH), right);
66              g.DrawString(c.ThanhTien.ToString("N0"), fText, Brushes.Black, new RectangleF(x[4], y,
                ↪ x[5] - x[4], RowH), right);
67              y += RowH; next++; drawn++;
68          }
69          y += 4; g.DrawLine(pen, bounds.Left, y, bounds.Right, y); y += 6;
70
71          if (next < hd.ChiTiet.Count)       // còn dòng: sang trang sau, tổng cộng in ở trang cuối
72          {
73              g.DrawString("... (còn tiếp ở trang sau)", fSmall, Brushes.Gray, new
                ↪ RectangleF(bounds.Left, y, w, 22), right);
74              return true;
75          }
76          g.DrawString("TỔNG CỘNG:", fBold, Brushes.Black, new RectangleF(bounds.Left, y, w * 0.6f,
                ↪ 24));
77          g.DrawString($"{hd.Tong:N0} đ", fBold, Brushes.DarkRed, new RectangleF(bounds.Left, y, w,
                ↪ 24), right); y += 40;
```

```
78          g.DrawString("Cảm ơn quý khách! Thuốc đã bán không đổi trả.", fSmall, Brushes.Black, new
            ↪  RectangleF(bounds.Left, y, w, 22), center);
79          return false;
```

6.8 Quản lý danh mục thuốc và ảnh

Trang Danh mục thuốc cho phép tìm nhanh (`BindingSource.Filter`, không truy vấn lại CSDL), sắp xếp theo cột, thêm/sửa/xóa (chỉ Quản lý kho), đổi trạng thái kê đơn (cả hai vai trò) và xem tồn theo lô. Ảnh thuốc được chọn bằng `OpenFileDialog`, nạp vào `Bitmap` rồi giải phóng ngay đối tượng tạm để không khóa tệp; chỉ *tên tệp* được lưu trong CSDL, ảnh nằm ở thư mục `Images/` cạnh tệp thực thi. Tệp ảnh chỉ được ghi *sau khi* bản ghi thuốc lưu thành công (tệp ảnh hỏng thì báo lỗi ngay, còn thêm thuốc trùng mã bị từ chối thì không ghi đè ảnh của thuốc đang có). Xóa thuốc đã có lô bị khóa ngoại từ chối và được báo bằng thông báo dễ hiểu.

6.9 Kiểm tra dữ liệu nhập

Mọi ô nhập số dùng một điều khiển `NumberBox` chỉ nhận chữ số, tự ép về khoảng hợp lệ khi rời ô (số lượng tối thiểu 1), nên giao diện không thể gửi số lượng sai xuống Presenter; các kiểm tra > 0 ở Presenter và Service là lớp bảo vệ thứ hai. Lỗi nhập liệu hiện ngay cạnh ô nhờ `ErrorProvider`. Ô giá (giá bán, giá nhập) được đọc theo văn hóa hiện hành *không nhận dấu phân nhóm*, nên giá trị như “1.5” trên máy vi-VN bị báo lỗi thay vì bị hiểu thầm thành 15. Khi lọc hóa đơn theo khoảng ngày sai, trang Hóa đơn hiện cảnh báo và giữ nguyên bảng đang hiển thị (không báo nhầm là mất kết nối).

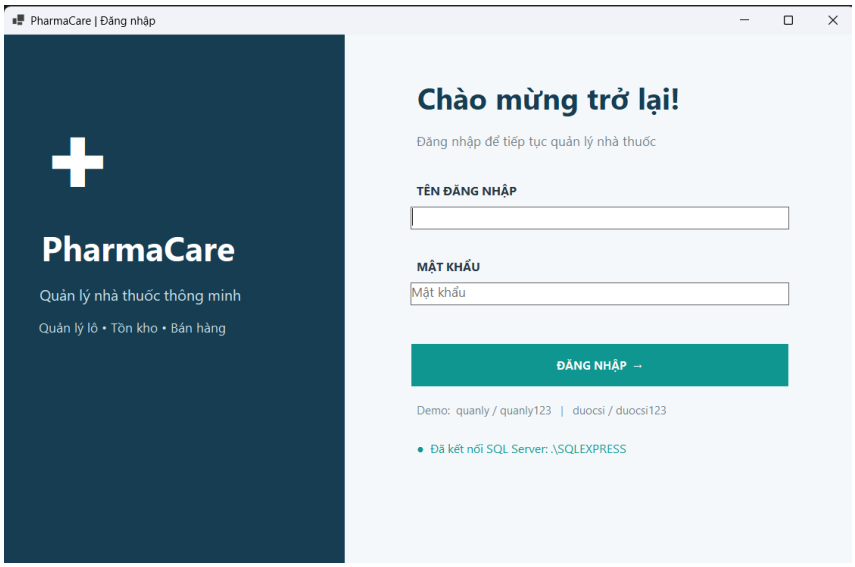
Chương 7

Thiết kế giao diện

Chương này giới thiệu từng màn hình của PharmaCare kèm ảnh chụp thực tế. Ứng dụng có một form chính dạng MDI; thanh bên (sidebar) chỉ hiện các chức năng mà vai trò đang đăng nhập được phép dùng, thanh trạng thái hiển thị người dùng, vai trò và giờ hệ thống.

7.1 Màn hình đăng nhập

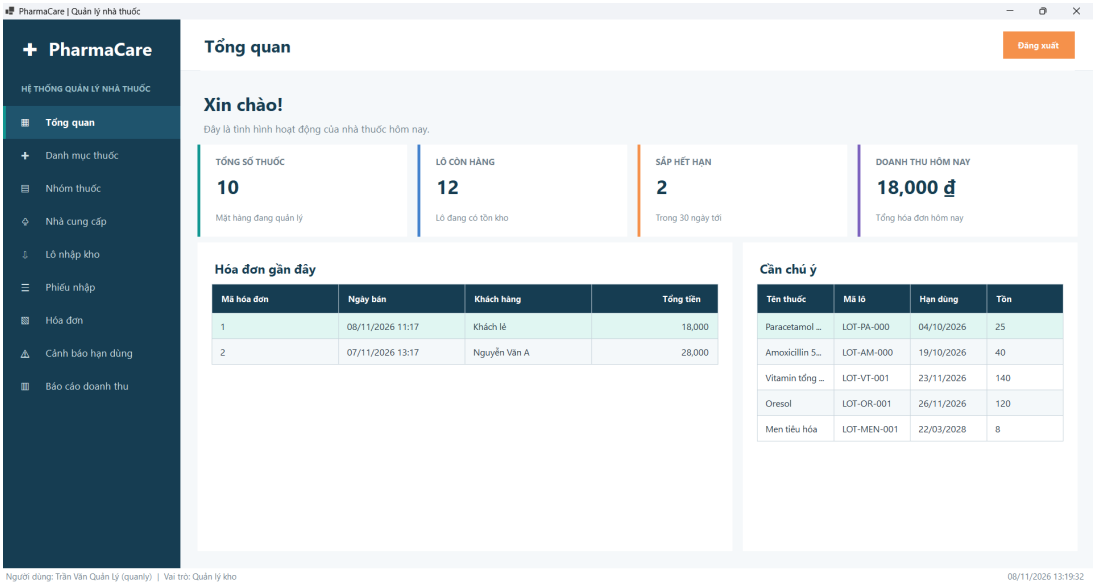
Người dùng nhập tên đăng nhập và mật khẩu. Nếu để trống, lỗi hiện ngay tại ô tương ứng; nếu sai, hệ thống báo cùng một thông báo cho cả sai tên và sai mật khẩu. Màn hình có gợi ý hai tài khoản demo và nút *Chi tiết lỗi* khi chưa kết nối được SQL Server.



Hình 7.1: Màn hình đăng nhập

7.2 Trang tổng quan

Hiển thị các số liệu chính và danh sách các lô cần chú ý (hết hạn, sắp hết hạn, sắp hết hàng) để người dùng nắm tình hình ngay khi vào ứng dụng.



Hình 7.2: Trang tổng quan

7.3 Danh mục thuốc

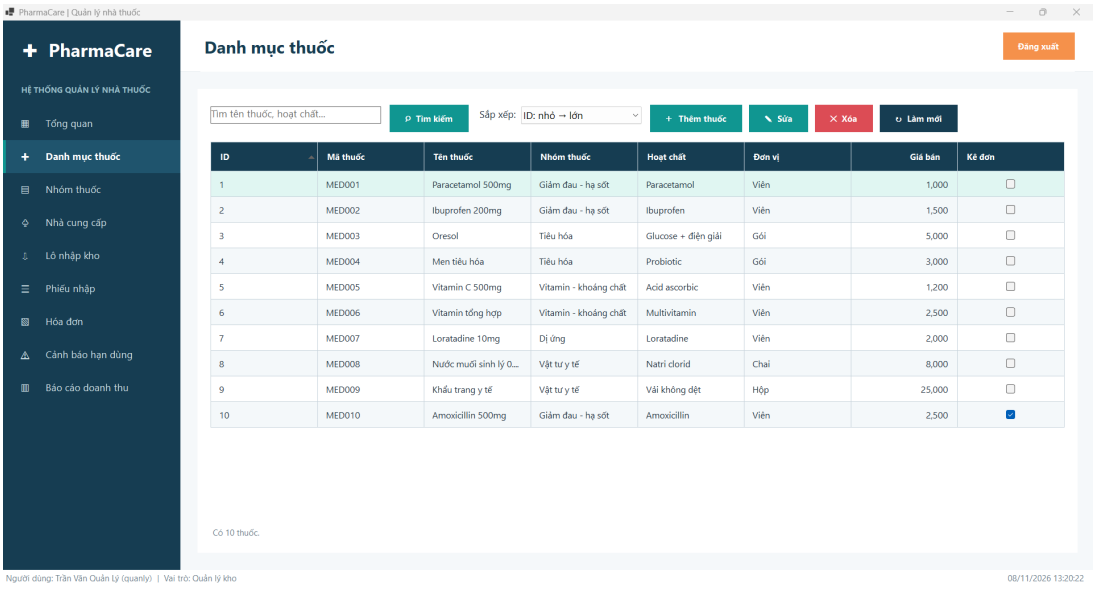
Bảng thuốc có thể sắp xếp theo cột, tìm nhanh theo từ khóa (lọc trên dữ liệu đã tải), xem ảnh thuốc và tồn theo lô. Quản lý kho có thêm các nút Thêm, Sửa, Xóa và menu chuột phải; Dược sĩ chỉ xem và đổi trạng thái kê đơn.

7.4 Thêm và sửa thuốc

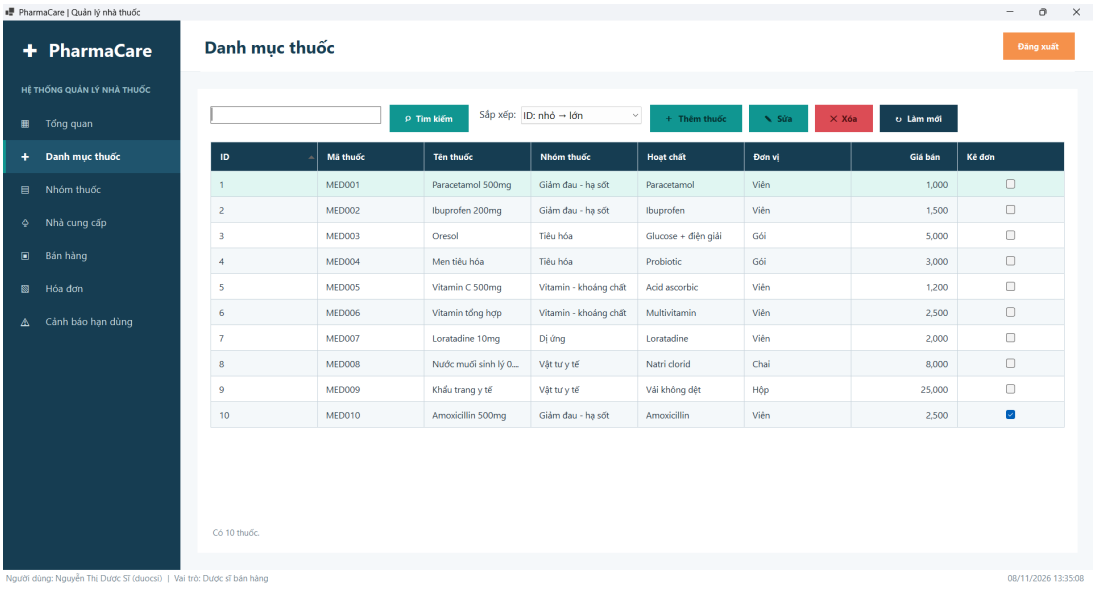
Biểu mẫu nhập mã, tên, nhóm, hoạt chất, đơn vị, giá niêm yết, kê đơn và ảnh. Dữ liệu sai (giá bằng 0 hoặc không phải số, tên trống, trùng mã) được báo ngay tại ô bằng `ErrorProvider`.

7.5 Nhóm thuốc và nhà cung cấp

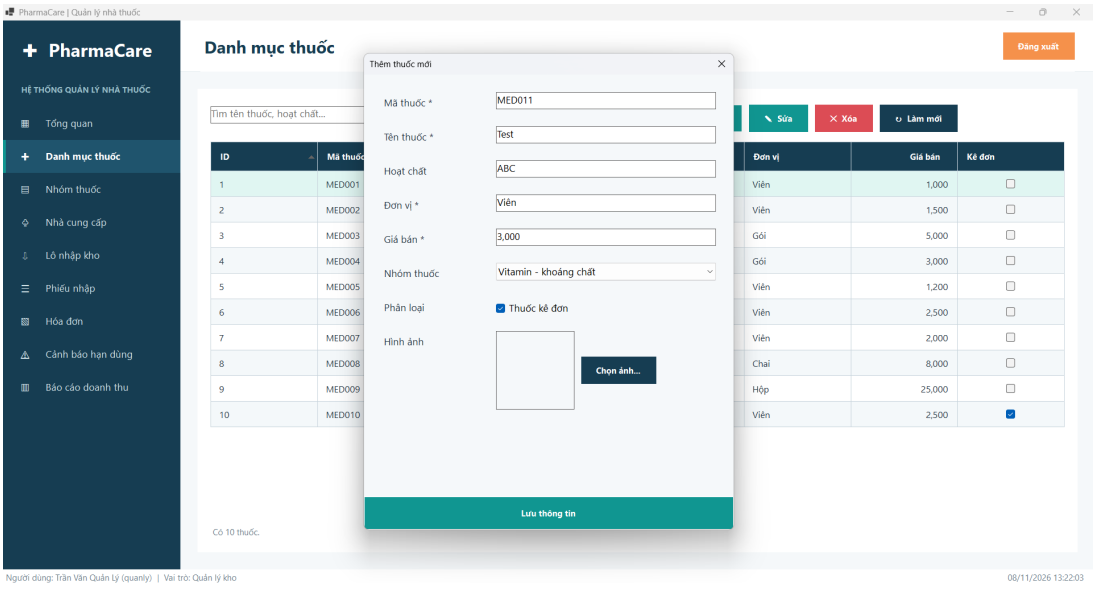
Hai trang tra cứu danh mục nhóm thuốc và nhà cung cấp; trang Nhóm thuốc cho phép đổi trạng thái kê đơn của thuốc.



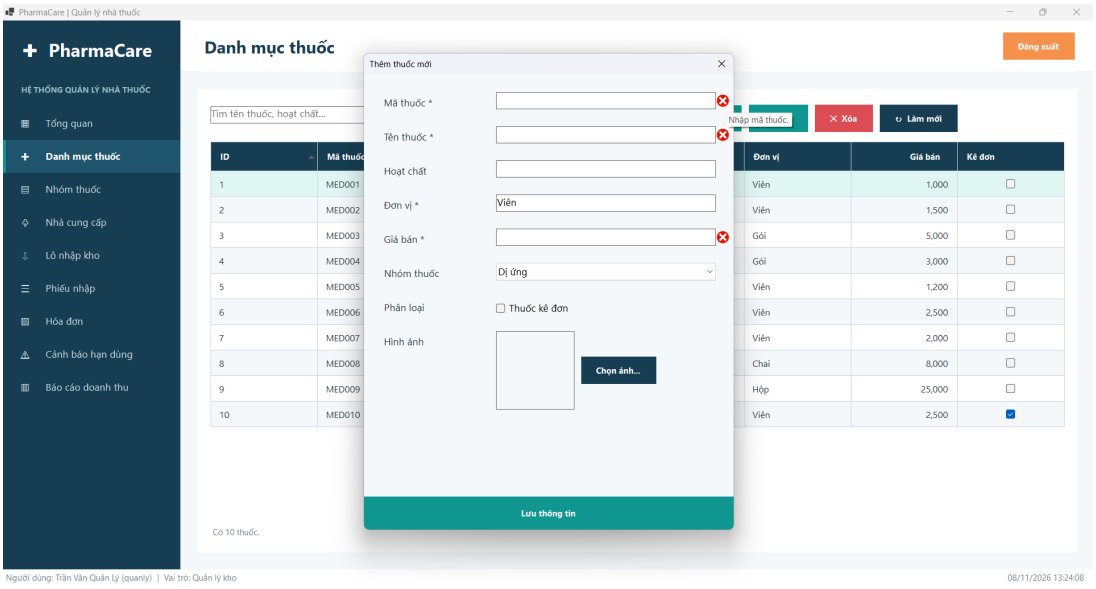
Hình 7.3: Danh mục thuốc (tài khoản Quản lý kho)



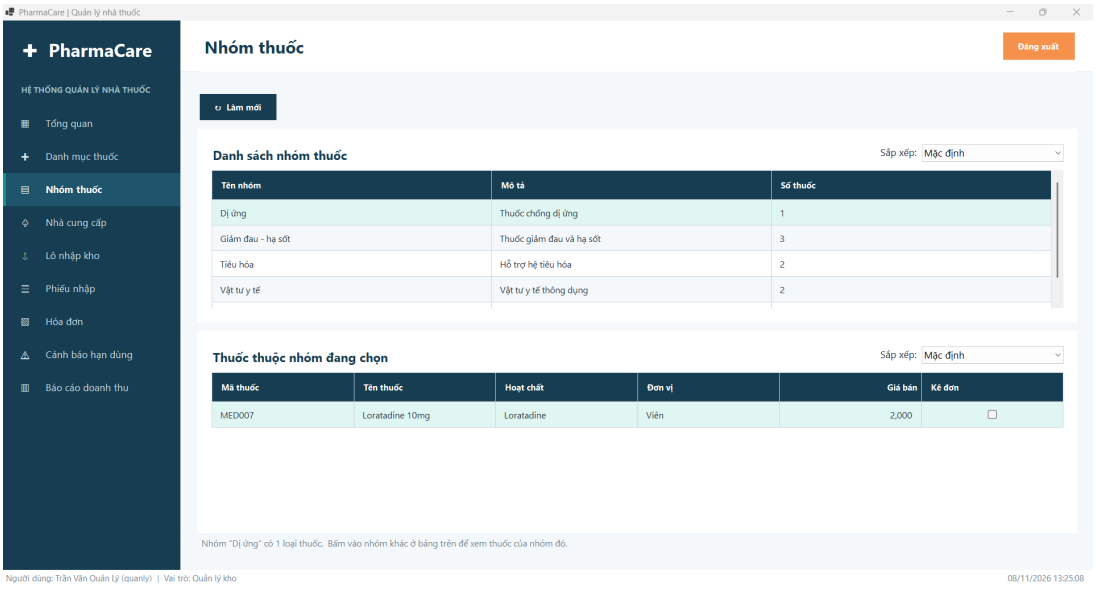
Hình 7.4: Danh mục thuốc với tài khoản Dược sĩ: các nút chỉnh sửa bị khóa



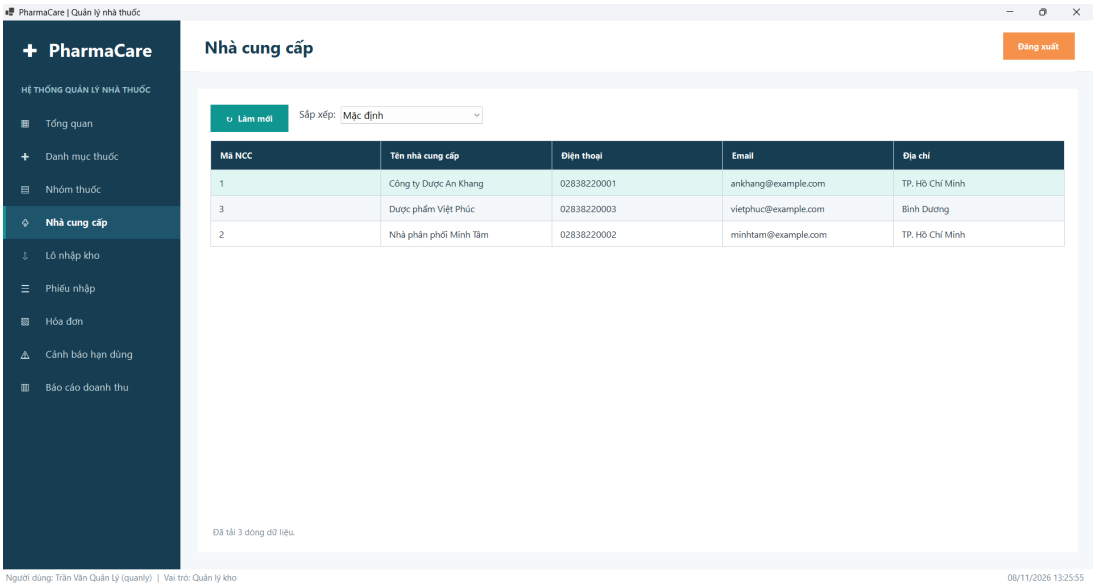
Hình 7.5: Biểu mẫu thêm thuốc



Hình 7.6: Báo lỗi nhập liệu ngay tại ô (`ErrorProvider`)



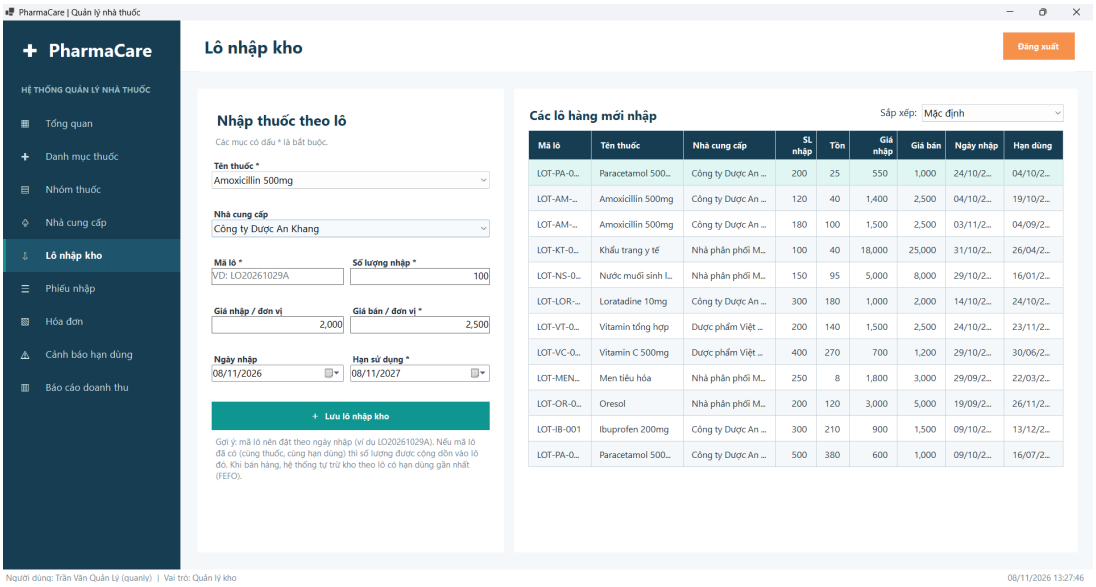
Hình 7.7: Trang Nhóm thuốc



Hình 7.8: Trang Nhà cung cấp

7.6 Nhập kho theo lô

Người dùng chọn thuốc (ô Giá bán tự điền giá niêm yết), nhà cung cấp, nhập mã lô, ngày nhập, hạn sử dụng, số lượng, giá nhập, giá bán. Bên phải là bảng các lô đã có để đối chiếu hạn dùng khi nhập lại lô cũ.



Hình 7.9: Trang Lô nhập kho

7.7 Lịch sử phiếu nhập

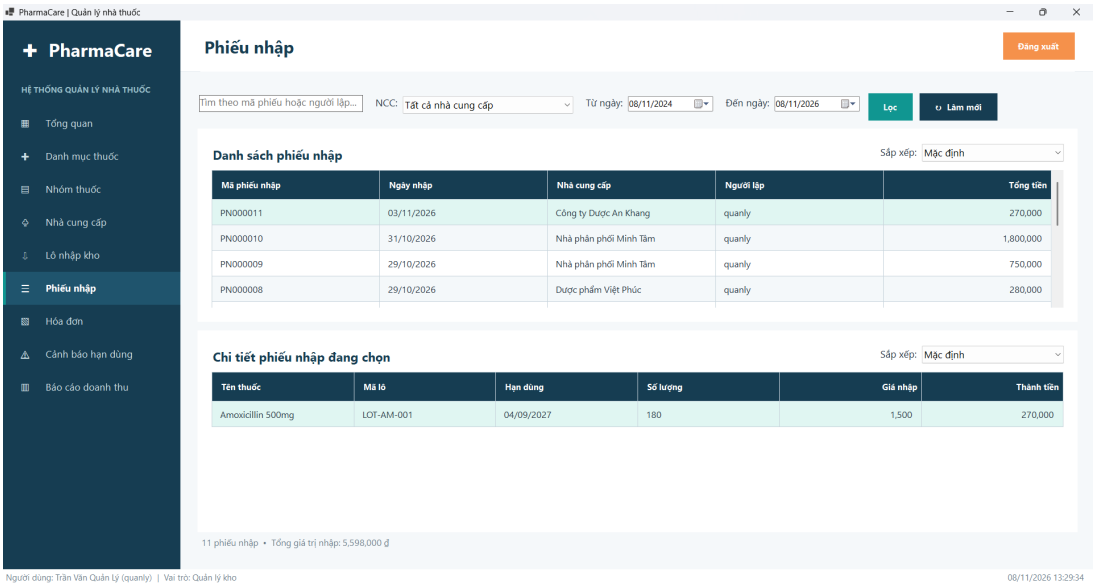
Trang *Phiếu nhập* (chỉ Quản lý kho) có bố cục giống trang Hóa đơn: phía trên là danh sách phiếu nhập, lọc theo từ khóa (mã phiếu hoặc người lập), nhà cung cấp và khoảng ngày; phía dưới là chi tiết các lô của phiếu đang chọn. Thanh trạng thái cho biết số phiếu và tổng giá trị nhập của kết quả lọc.

7.8 Bán hàng

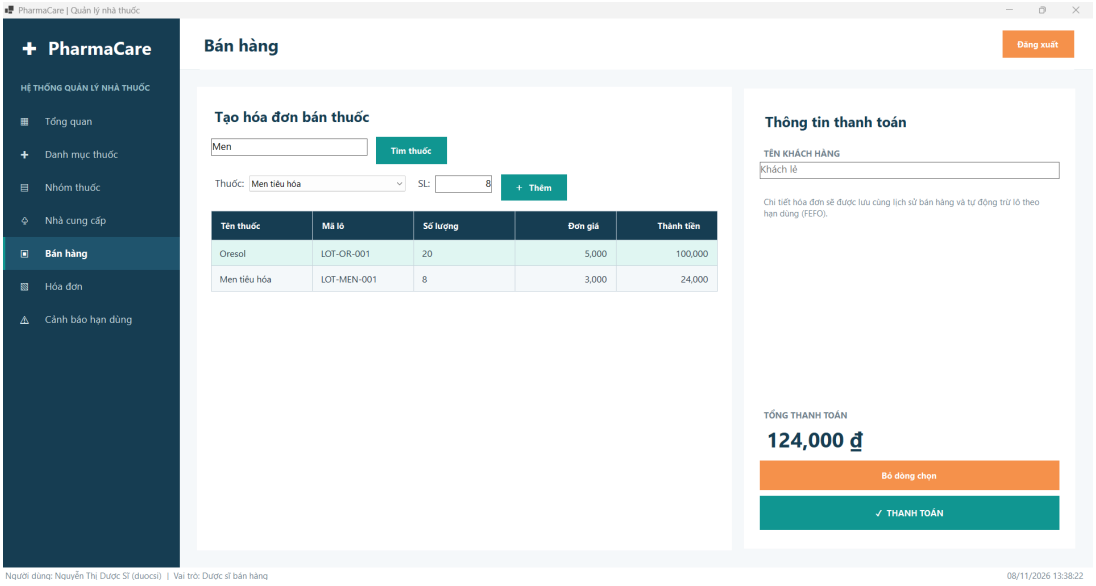
Dược sĩ tìm và chọn thuốc, nhập số lượng, bấm Thêm: hệ thống tự phân bổ theo FEFO nên một thuốc có thể sinh nhiều dòng (mỗi dòng một lô). Có thể bỏ dòng, nhập tên khách (để trống là *Khách lẻ*) rồi Thanh toán.

7.9 Xem trước và in hóa đơn

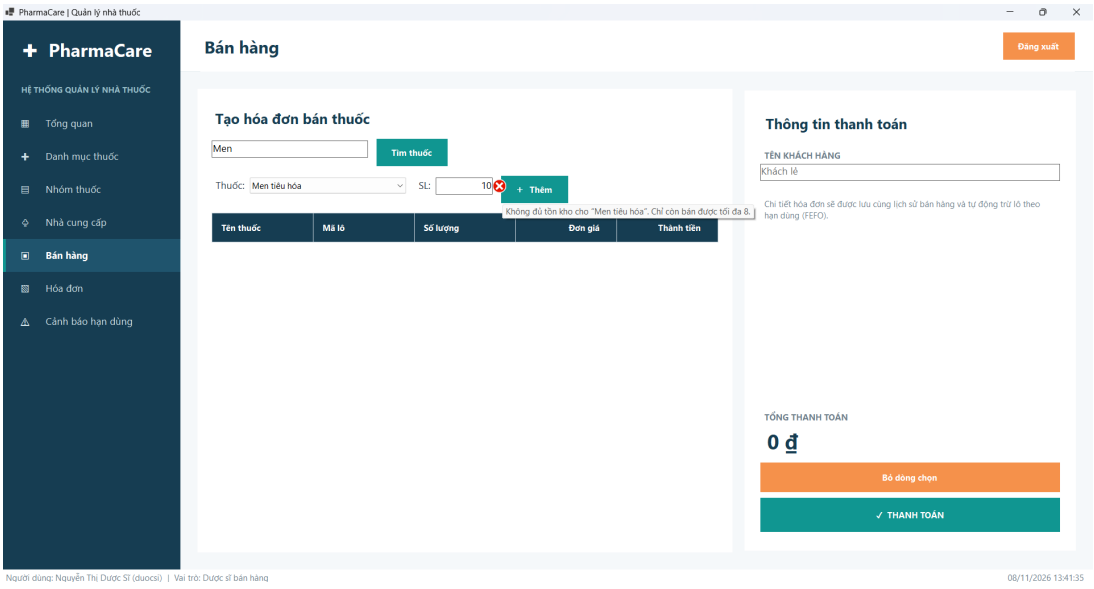
Sau khi thanh toán thành công, hệ thống hỏi có xem và in hóa đơn không; hóa đơn nhiều dòng tự sang trang và tổng cộng nằm ở trang cuối.



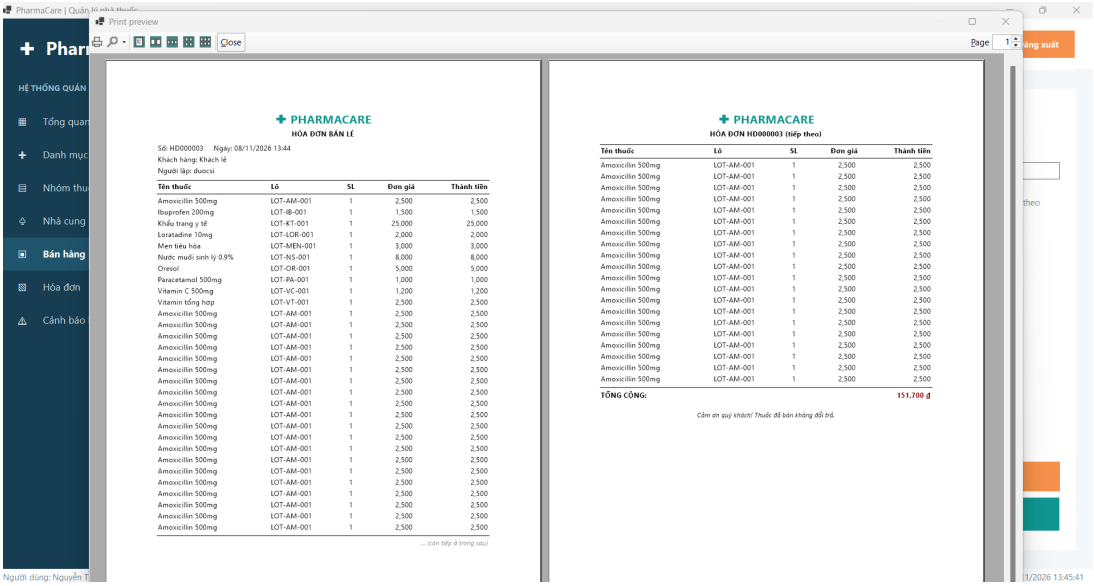
Hình 7.10: Trang Phiếu nhập (lịch sử nhập kho)



Hình 7.11: Trang Bán hàng với giỏ hàng phân bổ nhiều lô



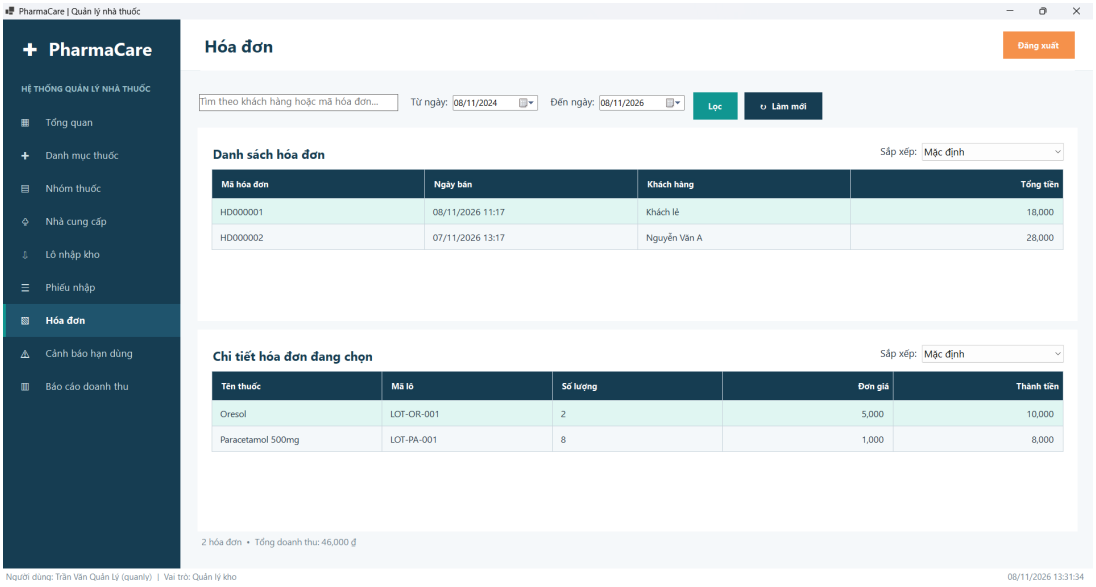
Hình 7.12: Báo lỗi khi bán vượt tồn: biểu tượng lỗi đỏ cạnh ô Số lượng, nội dung (“Chỉ còn bán được tối đa 8”) hiện khi rê chuột



Hình 7.13: Xem trước hóa đơn (Print Preview): hóa đơn nhiều dòng tự sang trang thứ hai, tổng cộng ở trang cuối

7.10 Tra cứu hóa đơn

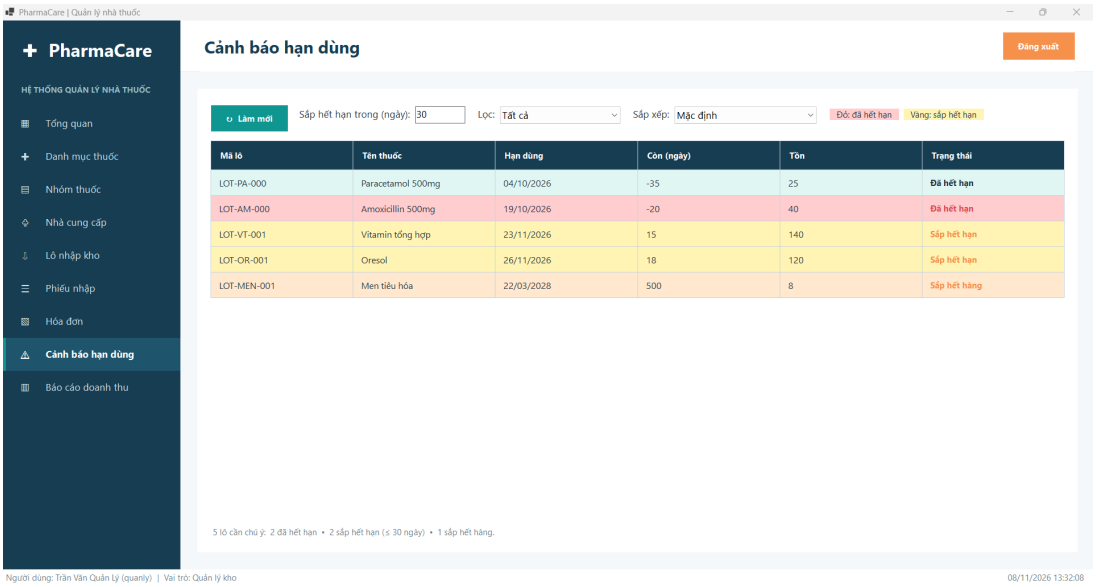
Lọc theo từ khóa (tên khách hoặc số hóa đơn) và khoảng ngày; chọn một hóa đơn để xem chi tiết các dòng bên dưới.



Hình 7.14: Trang Hóa đơn

7.11 Cảnh báo hạn dùng

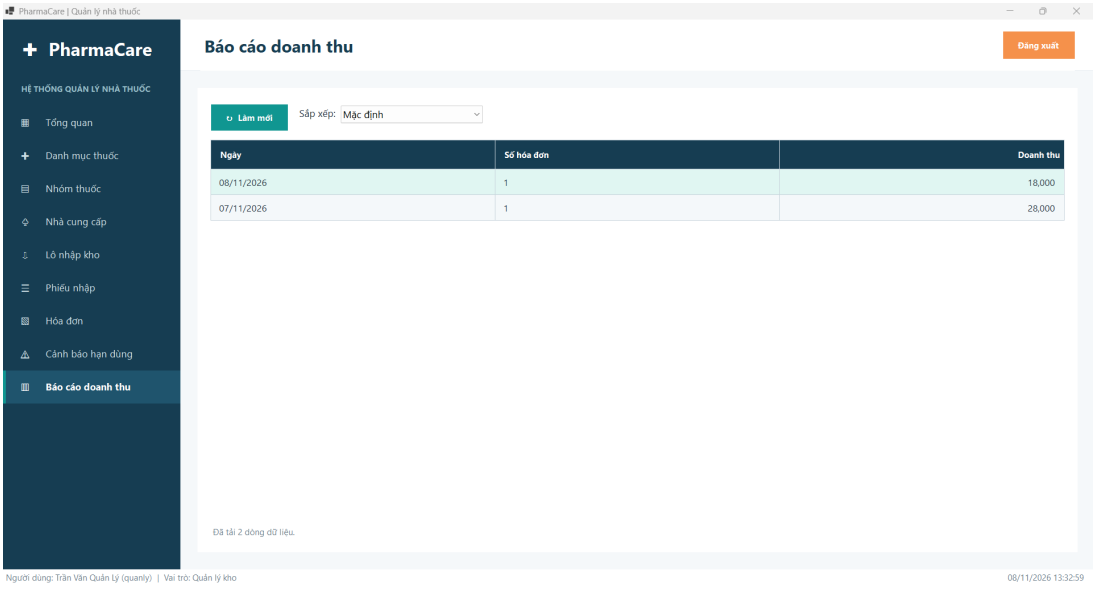
Bảng các lô cần chú ý tô màu theo mức độ, có bộ lọc theo trạng thái và tự cập nhật khi kho thay đổi.



Hình 7.15: Trang Cảnh báo hạn dùng với các lô tô màu

7.12 Báo cáo doanh thu

Dành cho Quản lý kho: tổng hợp doanh thu theo ngày từ các hóa đơn đã lập.



Hình 7.16: Báo cáo doanh thu theo ngày

Chương 8

Kiểm thử và kết quả

8.1 Môi trường và phương pháp

Kiểm thử chức năng (hộp đen) trên ứng dụng thật với cơ sở dữ liệu mẫu được tạo mới trước mỗi đợt test. Mỗi ca nêu rõ dữ liệu, thao tác và kết quả mong đợi; các ca bao gồm cả tình huống lỗi (sai mật khẩu, trùng mã, bán vượt tồn, thuốc hết hạn, nhập sai hạn dùng, rollback, lỗi máy in). Ngoài ra nhóm rà soát mã nguồn theo từng lớp để tìm lỗi logic mà ca kiểm thử chưa chạm tới (mục 8.4). **Lưu ý:** dữ liệu mẫu có hạn dùng tính theo ngày chạy script (ví dụ `LOT-OR-001` còn

Bảng 8.1: Môi trường kiểm thử

Hạng mục	Giá trị
Hệ điều hành	Windows 11 26H2
SQL Server	SQL Server 2022 Express (16.0.1000), instance `.\SQLEXPRESS`
.NET / IDE	.NET 10, Visual Studio 2026
Dữ liệu	`02_InsertSampleData.sql` chạy mới trước khi test
Người thực hiện / ngày	Cả nhóm (4 thành viên), 05–07/11/2026

18 ngày); sau khoảng hai tuần một số lô tự hết hạn nên các ca TC15 và TC19 cho kết quả khác. Vì vậy script được chạy lại ngay trước khi test, và các ca được chạy lần lượt từ TC01 đến TC20 trên cơ sở dữ liệu mới.

8.2 Bảng ca kiểm thử

Bảng 8.2 liệt kê 20 ca kiểm thử, mỗi ca một thao tác và một kết quả mong đợi. Cột *Kết quả* ghi *Đạt* hoặc *Không đạt* sau khi chạy thực tế.

Bảng 8.2: Bảng ca kiểm thử (20 ca)

ID	Chức năng	Thao tác / dữ liệu	Kết quả mong đợi	Kết quả
TC01	Đăng nhập – Dược sĩ	`duocsi` / `duocsi123`	Vào màn hình chính, sidebar 7 mục, không có Lô nhập kho	Đạt
TC02	Đăng nhập – Quản lý kho	`quanly` / `quanly123`	Vào màn hình chính, sidebar 9 mục, không có Bán hàng	Đạt
TC03	Đăng nhập – Sai mật khẩu	`duocsi` / `sai123`	Báo lỗi, không vào được	Đạt
TC04	Đăng nhập – Bảo mật	Tên đăng nhập `' OR 1=1 --`	Không đăng nhập được	Đạt
TC05	Thêm thuốc – Hợp lệ	Mã mới `MED011`, tên, giá 3000	Thêm thành công, lưới cập nhật	Đạt
TC06	Thêm thuốc – Trùng mã	Mã `MED001`	Báo lỗi trùng mã, không lưu	Đạt
TC07	Thêm thuốc – Dữ liệu rỗng	Để trống tên thuốc	Báo lỗi tại ô tên, không lưu	Đạt
TC08	Sửa thuốc	Đổi giá `MED011` thành 3500	Lưu thành công, lưới hiện giá mới	Đạt
TC09	Xóa thuốc – Mới tạo	Xóa `MED011`	Xóa thành công, thuốc biến mất khỏi lưới	Đạt
TC10	Xóa thuốc – Đã có lô nhập	Xóa `MED001`	Không cho xóa, hiện thông báo	Đạt
TC11	Tìm thuốc	Gõ `vitamin` vào ô tìm kiếm	Chỉ còn các thuốc Vitamin	Đạt
TC12	Phân quyền – Dược sĩ	`duocsi` mở Danh mục thuốc	Nút Thêm/Sửa/Xóa bị khóa	Đạt
TC13	Nhập kho – Lô mới	`quanly`, `MED001`, mã lô `LO-NEW-01`, SL 100, hạn +1 năm	Lô hiện trong bảng	Đạt
TC14	Phiếu nhập – Lọc	`quanly`: Phiếu nhập → nhà cung cấp Công ty Dược An Khang → Lọc	Chỉ còn phiếu của An Khang; chọn một phiếu thì thấy chi tiết lô	Đạt
TC15	Bán hàng – FEFO	`duocsi`: Oresol SL 20 → Thanh toán	Lấy từ `LOT-OR-001`, tồn giảm 20, có hóa đơn mới	Đạt

ID	Chức năng	Thao tác / dữ liệu	Kết quả mong đợi	Kết quả
TC16	Bán hàng – Vượt tồn	Men tiêu hóa (tồn 8): thêm SL 9, không thanh toán	Báo tối đa 8, giỏ không đổi	Đạt
TC17	In hóa đơn	Sau TC15: Print Preview hóa đơn vừa tạo	Hóa đơn đúng thuốc và số lượng, tổng cộng bằng tổng thanh toán	Đạt
TC18	Hóa đơn – Tra cứu	Trang Hóa đơn: gõ `Nguyễn Văn A` → Lọc	Chỉ còn hóa đơn của A, tổng 28,000	Đạt
TC19	Cảnh báo hạn dùng	Mở trang Cảnh báo	Có dòng đỏ (đã hết hạn), vàng (sắp hết hạn), cam (sắp hết hàng)	Đạt
TC20	Báo cáo doanh thu	`quanly` mở Báo cáo doanh thu	Doanh thu theo ngày, tổng khớp tổng các hóa đơn	Đạt

8.3 Kết quả thực hiện

Cả 20 ca TC01–TC20 đều *Đạt* khi chạy lần lượt trên cơ sở dữ liệu mới. Riêng TC16, thông báo "tối đa 8" hiển thị dưới dạng biểu tượng lỗi đỏ cạnh ô Số lượng (nội dung hiện khi rê chuột), không phải hộp thoại; giỏ hàng vẫn giữ nguyên đúng như mong đợi.

Bảng 8.3: Tổng hợp kết quả kiểm thử

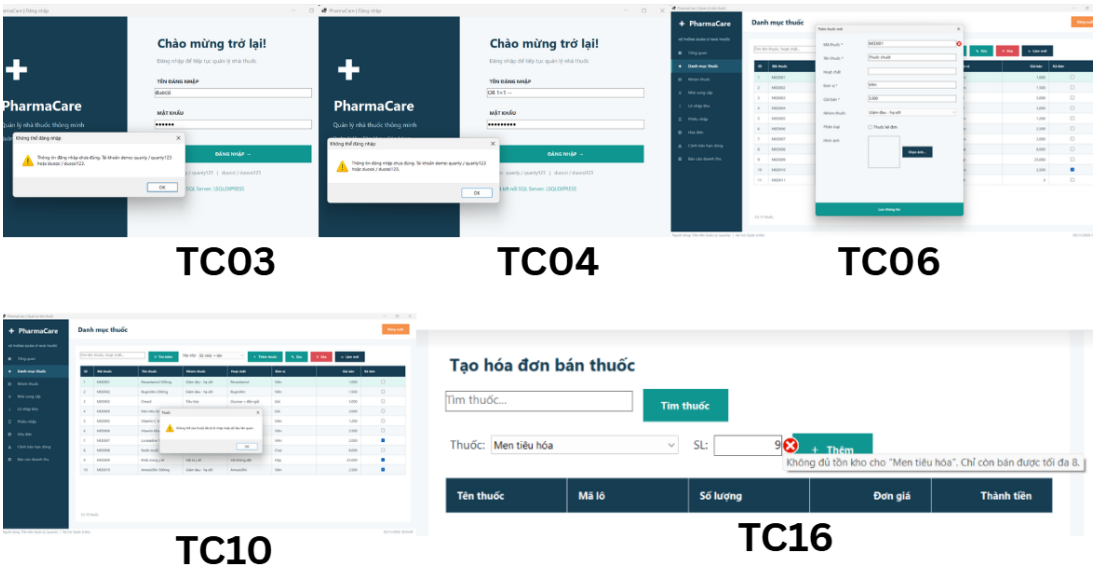
Chỉ tiêu	Số ca
Tổng số ca	20
Đạt	20
Không đạt (đã sửa và test lại đạt)	0
Chưa thực hiện	0

8.4 Các lỗi phát hiện khi rà soát và cách khắc phục

Qua rà soát mã nguồn và bộ ca kiểm thử, nhóm phát hiện và sửa các vấn đề sau (Bảng 8.4); các ca TC05–TC18 kiểm chứng lại những chức năng đã sửa này.

Bảng 8.4: Lỗi đã phát hiện và cách khắc phục

#	Vấn đề	Cách khắc phục
1	Lỗi ở bước xem trước/in sau khi đã lưu hóa đơn vẫn báo "thất bại, đã rollback"	Tách bước lưu và bước in trong `BanHangPresenter.ThanhToan`; lỗi in báo "đã lưu, không cần thanh toán lại"
2	Thanh toán thất bại do hết tồn nhưng giỏ vẫn giữ dòng lô cũ, kẹt các lần thử sau	Làm trống giỏ và nạp lại danh sách thuốc khi gặp `KhongDuTonKhoException`
3	Sửa giá trong Danh mục thuốc không ảnh hưởng giá bán; nhập lại lô ghi đè giá bán của lô	Giữ giá bán của lô khi cộng dồn; giá nhập bình quân gia quyền; điền sẵn giá niêm yết ở ô Giá bán khi nhập
4	Hóa đơn hơn khoảng 35 dòng bị cắt khi in	Phân trang trong `InvoiceRenderer`, tổng cộng ở trang cuối
5	Lỗi "hạn dùng trước ngày nhập" hiện nhầm ở ô Mã lô; thông báo trùng mã lô để trống mã	Ngoại lệ riêng gắn đúng ô Hạn sử dụng; điền mã lô vào thông báo
6	Thời gian chờ kết nối 4 giây quá ngắn với LocalDB; thiếu thủ tục lưu sẵn làm tạo lại toàn bộ bảng	Tăng thời gian chờ; chỉ chạy lại script thủ tục khi thiếu thủ tục, giữ dữ liệu
7	Quyền chỉ được kiểm tra ở giao diện	Thêm `AppSession.Require` ở các Service
8	Tìm hóa đơn không thoát ký tự %, _, [; khoảng ngày sai không được báo	`EscapeLike` trong Repository; kiểm tra "Từ ngày $\leq$ Đến ngày"
9	Truy vấn chi tiết hóa đơn chạy hai lần; lỗi tải danh mục bị nuốt im lặng	Ghi nhớ hóa đơn đang hiển thị; hiện thông báo lỗi kết nối
10	Dữ liệu mẫu: tổng hóa đơn của Nguyễn Văn A là 32 000 nhưng chi tiết chỉ 28 000	Sửa dữ liệu mẫu thành 28 000 để báo cáo doanh thu khớp chi tiết
11	Thêm thuốc trùng mã (có chọn ảnh) vẫn ghi đè tệp ảnh của thuốc đang có	Đọc thử ảnh trước, chỉ ghi tệp `Images/` sau khi lưu CSDL thành công (`ThuocService.Luu`)
12	Khoảng ngày sai ở trang Hóa đơn xóa bảng và báo "chưa kết nối được cơ sở dữ liệu"	Thêm `IHoaDonView.ShowWarning`: chỉ cảnh báo, giữ nguyên dữ liệu đang hiển thị
13	Trang Nhập kho mất thuốc/giá đang chọn khi tải lại danh sách; ngày nhập sai bị tự đổi thành hôm nay	Giữ lại lựa chọn và giá khi nạp lại; báo lỗi tại ô Ngày nhập
14	Ô giá nhận dấu phân nhóm nên "1.5" bị đọc thành 15; tìm hóa đơn không khớp mã `HDxxxxxx`	Chỉ nhận dấu thập phân; tìm theo cả mã hóa đơn hiển thị



Hình 8.1: Minh chứng một số ca kiểm thử: TC03 (sai mật khẩu), TC04 (thử SQL injection), TC06 (trùng mã), TC10 (không cho xóa thuốc đã có lô) và TC16 (bán vượt tồn)

8.5 Đánh giá

Các chức năng chính (đăng nhập và phân quyền, quản lý thuốc, nhập kho theo lô, bán hàng theo FEFO, in hóa đơn, cảnh báo, báo cáo doanh thu) chạy đúng trong cả 20 ca kiểm thử, và các lỗi tìm thấy khi rà soát mã nguồn đã được sửa (Bảng 8.4).

Điểm còn yếu: bộ kiểm thử gồm 20 ca chạy thủ công, chưa có kiểm thử tự động. Thông báo vượt tồn ở TC16 chỉ hiện khi rê chuột vào biểu tượng lỗi nên người dùng mới có thể bỏ sót. Ứng dụng cũng chưa có màn hình đổi mật khẩu và chưa có trường thông tin sử dụng cho thuốc; đây là hướng phát triển tiếp theo.

Chương 9

Kết luận, hạn chế và hướng phát triển

9.1 Kết quả đạt được

Nhóm đã xây dựng ứng dụng PharmaCare đáp ứng các yêu cầu đề ra:

- Cơ sở dữ liệu gồm chín bảng với đầy đủ ràng buộc toàn vẹn, hai chỉ mục và ba thủ tục lưu sẵn; ứng dụng tự tạo cơ sở dữ liệu mẫu ở lần chạy đầu.
- Kiến trúc MVP kết hợp nhiều lớp (View – Presenter – Service – Repository), mỗi lớp phụ thuộc lớp dưới qua giao diện.
- Quản lý tồn kho theo lô, bán theo FEFO (không bán lô hết hạn, không bán vượt tồn kể cả phần đang nằm trong giỏ).
- Giao dịch bán và nhập kho có rollback; trừ kho có điều kiện nên an toàn khi nhiều quầy cùng bán.
- Phân quyền hai vai trò ở cả giao diện và tầng nghiệp vụ; mật khẩu băm PBKDF2-SHA256; truy vấn tham số hóa.
- Cảnh báo hạn dùng tô màu và tự cập nhật; in hóa đơn nhiều trang; tra cứu lịch sử nhập kho và lịch sử bán hàng; báo cáo doanh thu.
- Bộ 20 ca kiểm thử và quy trình rà soát mã nguồn giúp phát hiện và sửa các lỗi nêu ở Chương 8.

9.2 Hạn chế

- Trang Nhóm thuốc và Nhà cung cấp mới chủ yếu để tra cứu (chưa có đầy đủ thêm/sửa/xóa); chưa có quản lý tài khoản người dùng.
- Thông báo đăng nhập sai có kèm tài khoản demo – phù hợp đồ án nhưng cần bỏ khi triển khai thật; số vòng lặp PBKDF2 (100 000) nên tăng theo khuyến nghị hiện hành [10].

- Chưa có bộ kiểm thử tự động (unit test) và chưa đo hiệu năng với dữ liệu lớn.
- Ứng dụng dùng một CSDL cục bộ, chưa hỗ trợ nhiều chi nhánh và chưa có nhật ký thao tác.

9.3 Hướng phát triển

- Hoàn thiện CRUD nhóm thuốc, nhà cung cấp và quản lý tài khoản; nhật ký thao tác (audit log).
- Bổ sung unit test cho `BanHangService` và `NhapKhoService` (đã tách giao diện Repository và cho phép tiêm hàm lấy ngày hiện tại).
- Quét mã vạch, xuất Excel/PDF báo cáo, báo cáo theo tháng và theo thuốc.
- Chuyển sang kiến trúc client–server (Web API) để dùng nhiều quầy và nhiều chi nhánh.

Phụ lục A

Phụ lục cài đặt, tài khoản demo và script cơ sở dữ liệu

A.1 Yêu cầu

Windows 10/11; Visual Studio 2026 với workload *.NET desktop development* và .NET 10 SDK; SQL Server Express/Developer hoặc LocalDB (kèm SSMS nếu muốn xem dữ liệu).

A.2 Các bước

1. Tải mã nguồn từ GitHub (`https://github.com/cubo40925-prog/Nhom06_QuanLyNhaThuoc`) hoặc giải nén gói nộp bài.
2. Mở `PharmaCare.sln` bằng Visual Studio, chọn cấu hình Debug và nhấn `F5`.
3. Lần chạy đầu, ứng dụng tự dò SQL Server, tạo cơ sở dữ liệu `PharmaCareDB` và nạp dữ liệu mẫu.
4. Đăng nhập bằng một trong hai tài khoản demo trong Bảng A.1.

Bảng A.1: Tài khoản demo

Tài khoản	Mật khẩu	Vai trò
`duocsi`	`duocsi123`	Dược sĩ bán hàng
`quanly`	`quanly123`	Quản lý kho

A.3 Chạy script thủ công

Nếu tự động tạo thất bại, mở SSMS và chạy lần lượt `Database/01_CreateDatabase.sql`, `02_InsertSampleData.sql`, `03_StoredProcedures.sql`. Để sao lưu, dùng `04_BackupDatab`

`ase.sql` (đổi đường dẫn tệp `.bak` cho phù hợp máy).

A.4 Sự cố thường gặp

- *Không kết nối được SQL Server*: bấm *Chi tiết lỗi* trên màn hình đăng nhập, bảo đảm dịch vụ SQL Server hoặc LocalDB đang chạy.
- *Báo thiếu SDK khi build*: cài .NET 10 SDK và dùng Visual Studio 2026.
- *Không in được hóa đơn*: cài một máy in mặc định (kể cả *Microsoft Print to PDF*); hóa đơn vẫn được lưu và có thể in lại.

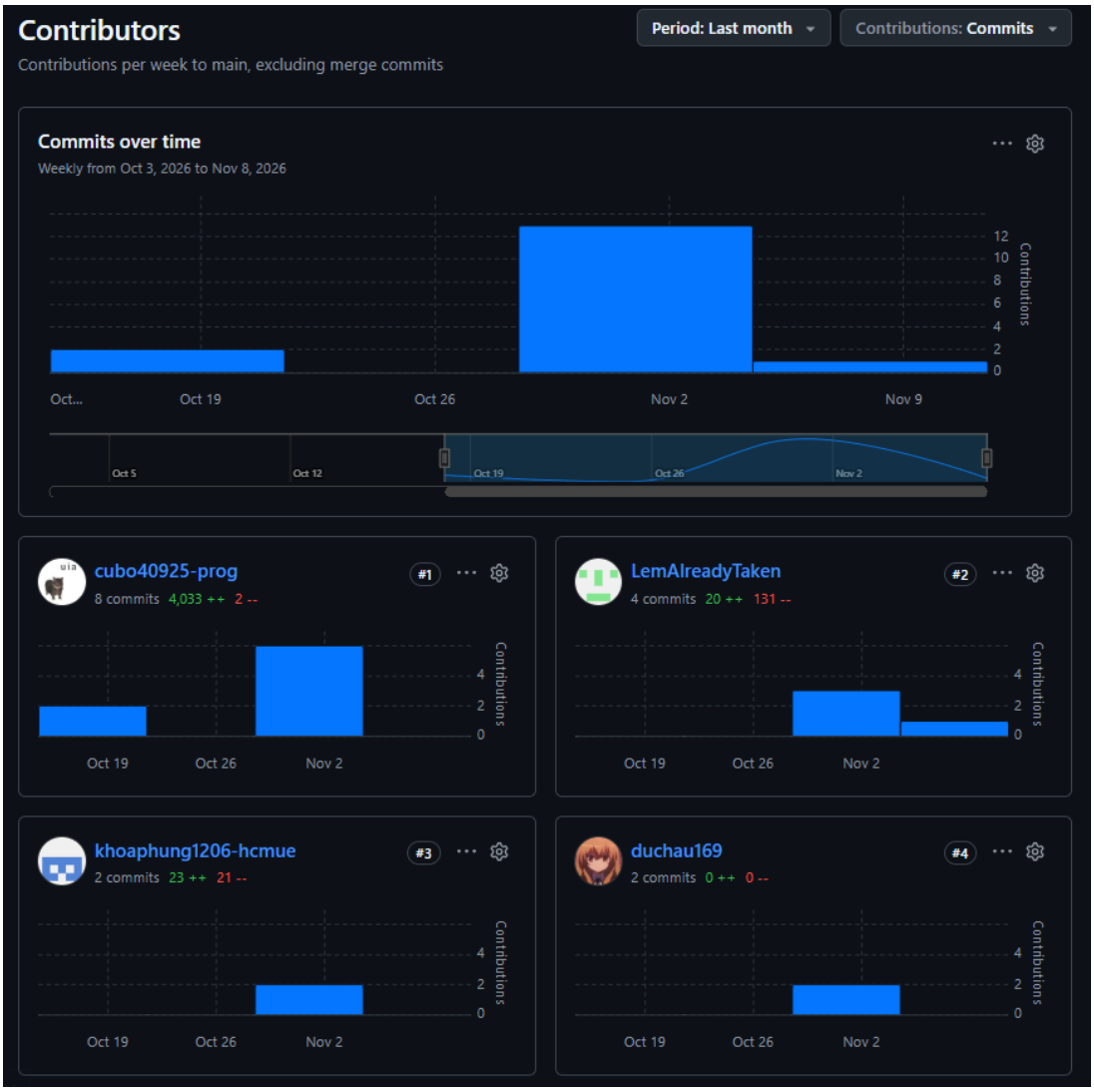
Phụ lục B

Đóng góp của các thành viên

Kho mã nguồn chung: https://github.com/cubo40925-prog/Nhom06_QuanLyNhaThuoc. Mỗi thành viên commit phần việc của mình bằng tài khoản GitHub riêng; lịch sử commit là minh chứng đóng góp.

Bảng B.1: Phân công công việc

Thành viên	MSSV	Công việc phụ trách	Tài khoản GitHub
Nguyễn Đan Trường	51.01.104.110	Thuyết trình; viết code; QA; báo cáo LaTeX và PDF	LemAlreadyTaken
Phùng Đăng Khoa	51.01.104.046	Chạy test case; file Excel kết quả thực tế; viết code; báo cáo LaTeX và PDF	khoaphung1206-hcmue
Nguyễn Trọng Đức Hậu	51.01.104.025	Khởi tạo database; viết code; file Excel kết quả thực tế	duchau169
Lê Quốc Tuấn	51.01.104.115	Quản lý GitHub; slide PowerPoint; viết code; báo cáo LaTeX và PDF	cubo40925-prog



Hình B.1: Thống kê commit của các thành viên trên GitHub

Phụ lục C

Mã nguồn tiêu biểu

Phụ lục này liệt kê nguyên văn các tệp quan trọng nhất (toàn bộ mã nằm trong kho GitHub).

C.1 Tầng Service: `BanHangService.cs`

Danh sách mã C.1: BanHangService.cs – FEFO và thanh toán

```
using System.Data;
using PharmaCare.Exceptions;
using PharmaCare.Helpers;
using PharmaCare.Models;
using PharmaCare.Repositories;
namespace PharmaCare.Services;

public sealed class BanHangService
{
    readonly IBanHangRepository _repo;
    readonly Func<DateTime> _homNay;

    public BanHangService(IBanHangRepository repo, Func<DateTime>? homNay = null)
    {
        _repo = repo;
        _homNay = homNay ?? (() => DateTime.Today);
    }

    public DataTable GetThuocBan() => _repo.GetThuocBan();

    /// <summary>
    /// FEFO: bán lô còn hạn có hạn dùng gần nhất trước, tràn sang lô kế tiếp nếu chưa đủ.
    /// Tính cả phần đã giữ chỗ trong giỏ để thêm cùng một thuốc nhiều lần không bán vượt tồn.
    /// </summary>
    public IReadOnlyList<ChiTietHoaDon> PhanBoFefo(int maThuoc, int soLuong,
    ↪  IEnumerable<ChiTietHoaDon> gio)
    {
        if (soLuong <= 0) throw new NghiepVuException("Số lượng phải lớn hơn 0.");
        string ten = _repo.LayTenThuoc(maThuoc) ?? throw new NghiepVuException("Không tìm thấy
        ↪  thuốc đã chọn.");

        var daGiu = gio.GroupBy(c => c.MaLo).ToDictionary(g => g.Key, g => g.Sum(c => c.SoLuong));
        var cacLo = _repo.GetLoConTon(maThuoc);
```

```
32          var homNay = _homNay();
33          var loBanDuoc = cacLo.Where(l => !l.DaHetHan(homNay)).OrderBy(l => l.HanDung).ThenBy(l =>
            ↪  l.MaLo).ToList();
34
35          if (loBanDuoc.Count == 0)
36          {
37              if (cacLo.Count > 0) throw new ThuocHetHanException(ten);          // chỉ còn lô hết
                ↪  hạn
38              throw new KhongDuTonKhoException(ten, 0);
39          }
40
41          var ketQua = new List<ChiTietHoaDon>();
42          int can = soLuong, tongConLai = 0;
43          foreach (var lo in loBanDuoc)
44          {
45              int trong = lo.SoLuongTon - daGiu.GetValueOrDefault(lo.MaLo);
46              if (trong <= 0) continue;
47              tongConLai += trong;
48              if (can == 0) continue;
49              int lay = Math.Min(trong, can);
50              ketQua.Add(new ChiTietHoaDon(maThuoc, ten, lo.MaLo, lo.SoLo, lay, lo.GiaBan));
51              can -= lay;
52          }
53          if (can > 0) throw new KhongDuTonKhoException(ten, tongConLai);
54          return ketQua;
55      }
56
57      public HoaDon ThanhToan(string tenKhach, string nguoiLap, IReadOnlyList<ChiTietHoaDon> gio)
58      {
59          AppSession.Require(Quyen.BanHang);
60          if (gio.Count == 0) throw new NghiepVuException("Vui lòng thêm ít nhất một thuốc vào hóa
            ↪  đơn.");
61          var hd = new HoaDon
62          {
63              TenKhach = string.IsNullOrWhiteSpace(tenKhach) ? "Khách lẻ" : tenKhach.Trim(),
64              NguoiLap = nguoiLap,
65              ChiTiet = gio.ToList()
66          };
67          hd.MaHD = _repo.LuuHoaDon(hd);       // transaction + trừ kho có điều kiện
68          hd.NgayLap = DateTime.Now;
69          AppEvents.BaoKhoThayDoi();
70          return hd;
71      }
72  }
```

C.2 Tầng Repository: `BanHangRepository.cs`

Danh sách mã C.2: BanHangRepository.cs – giao dịch bán hàng

```
1   using System.Data;
2   using Microsoft.Data.SqlClient;
3   using PharmaCare.Exceptions;
4   using PharmaCare.Models;
5   namespace PharmaCare.Repositories;
```

```
6
7   public sealed class BanHangRepository : IBanHangRepository
8   {
9       public DataTable GetThuocBan() => Db.Query("SELECT MedicineID,MedicineName FROM dbo.Medicines
        ↪  ORDER BY MedicineName");
10
11      public string? LayTenThuoc(int maThuoc) =>
12          Convert.ToString(Db.Scalar("SELECT MedicineName FROM dbo.Medicines WHERE MedicineID=@id",
            ↪  Db.P("@id", maThuoc)));
13
14      public IList<LoThuoc> GetLoConTon(int maThuoc)
15      {
16          var dt = Db.Query("SELECT
            ↪  BatchID,BatchCode,MedicineID,ExpiryDate,QuantityRemaining,SellingPrice FROM dbo.Batches
            ↪  WHERE MedicineID=@id AND QuantityRemaining>0 ORDER BY ExpiryDate,ReceivedDate,BatchID",
17                          Db.P("@id", maThuoc));
18          return dt.Rows.Cast<DataRow>().Select(r => new LoThuoc
19          {
20              MaLo = (int)r["BatchID"], SoLo = (string)r["BatchCode"], MaThuoc =
                ↪  (int)r["MedicineID"],
21              HanDung = (DateTime)r["ExpiryDate"], SoLuongTon = (int)r["QuantityRemaining"], GiaBan =
                ↪  (decimal)r["SellingPrice"]
22          }).ToList();
23      }
24
25      /// <summary>Bán thuốc trong MỘT transaction: Invoices -> mỗi dòng: trừ kho có điều kiện ->
        ↪  InvoiceLines.
26      /// Số dòng bị ảnh hưởng = 0 hoặc lỗi bất kỳ => Rollback toàn bộ.</summary>
27      public int LuuHoaDon(HoaDon hd)
28      {
29          using var cn = Db.OpenConnection();
30          using var tx = cn.BeginTransaction();
31          try
32          {
33              int maHD;
34              using (var cmd = new SqlCommand("INSERT
                ↪  dbo.Invoices(InvoiceDate,CustomerName,TotalAmount,CreatedBy) OUTPUT
                ↪  INSERTED.InvoiceID VALUES(GETDATE(),@name,@total,@user)", cn, tx))
35              {
36                  cmd.Parameters.Add(Db.P("@name", hd.TenKhach));
37                  cmd.Parameters.Add(Db.M("@total", hd.Tong));
38                  cmd.Parameters.Add(Db.P("@user", hd.NguoiLap));
39                  maHD = Convert.ToInt32(cmd.ExecuteScalar());
40              }
41              foreach (var c in hd.ChiTiet)
42              {
43                  using var tru = new SqlCommand(
44                      "UPDATE dbo.Batches SET QuantityRemaining=QuantityRemaining-@sl WHERE
                        ↪  BatchID=@ma AND QuantityRemaining>=@sl AND ExpiryDate>=CAST(GETDATE() AS
                        ↪  date)", cn, tx);
45                  tru.Parameters.Add(Db.P("@sl", c.SoLuong));
46                  tru.Parameters.Add(Db.P("@ma", c.MaLo));
47                  if (tru.ExecuteNonQuery() == 0) throw new KhongDuTonKhoException(c.TenThuoc, 0);
48
49                  using var ct = new SqlCommand("INSERT
                    ↪  dbo.InvoiceLines(InvoiceID,BatchID,Quantity,UnitPrice)
                    ↪  VALUES(@hd,@lo,@sl,@gia)", cn, tx);
50                  ct.Parameters.Add(Db.P("@hd", maHD));
51                  ct.Parameters.Add(Db.P("@lo", c.MaLo));
```

```
                ct.Parameters.Add(Db.P("@sl", c.SoLuong));
                ct.Parameters.Add(Db.M("@gia", c.DonGia));
                ct.ExecuteNonQuery();
            }
            tx.Commit();
            return maHD;
        }
        catch { try { tx.Rollback(); } catch { /* kết nối đã đứt */ } throw; }
    }
}
```

C.3 Presenter: BanHangPresenter.cs

Danh sách mã C.3: BanHangPresenter.cs

```
using PharmaCare.Exceptions;
using PharmaCare.Models;
using PharmaCare.Services;
using PharmaCare.Views;
namespace PharmaCare.Presenters;

public sealed class BanHangPresenter : PresenterBase
{
    readonly IBanHangView _view;
    readonly BanHangService _service;
    readonly string _nguoiLap;
    readonly GioHang _gio = new();

    public BanHangPresenter(IBanHangView view, BanHangService service, string nguoiLap)
    {
        _view = view; _service = service; _nguoiLap = nguoiLap;
        _gio.DaThayDoi += () => _view.ShowGioHang(_gio.Dong, _gio.Tong);      // event: giỏ đổi ->
        ↪  cập nhật View
        _view.ThemRequested += (_, _) => Them();
        _view.BoDongRequested += (_, _) => _gio.BoDong(_view.DongDangChon);
        _view.ThanhToanRequested += (_, _) => ThanhToan();
    }

    public void Start() => Chay(() => _view.ShowThuoc(_service.GetThuocBan()),
                                msg => _view.ShowError("Không tải được danh mục thuốc. Kiểm tra cơ
                                ↪  sở dữ liệu.\n" + msg));

    void Them()
    {
        _view.ShowSoLuongError("");
        if (!int.TryParse(_view.MaThuocDangChon, out int maThuoc)) { _view.ShowError("Hãy chọn một
        ↪  thuốc."); return; }
        if (!int.TryParse(_view.SoLuong, out int sl) || sl <= 0) { _view.ShowSoLuongError("Số lượng
        ↪  phải là số nguyên > 0."); return; }
        Chay(() => _gio.Them(_service.PhanBoFefo(maThuoc, sl, _gio.Dong)), _view.ShowError, ex =>
        ↪  _view.ShowSoLuongError(ex.Message));
    }

    void ThanhToan()
```

```
35      {
36          // Bước 1: LƯU hóa đơn (transaction). Chỉ lỗi ở bước này mới được báo là "đã rollback".
37          HoaDon hd;
38          try { hd = _service.ThanhToan(_view.TenKhach, _nguoiLap, _gio.Dong.ToList()); }
39          catch (KhongDuTonKhoException ex)
40          {
41              // Tồn kho đã đổi (máy khác bán trước...): các dòng trong giỏ trỏ vào lô cũ nên không
                ↪  còn đúng -> làm trống giỏ + nạp lại danh sách.
42              _gio.Xoa(); Start();
43              _view.ShowError("Thanh toán thất bại; dữ liệu đã được rollback, chưa có hóa đơn nào
                ↪  được lưu.\n" + ex.Message +
44                              "\n\nGiỏ hàng đã được làm trống vì tồn kho đã thay đổi. Vui lòng thêm
                                ↪  lại thuốc.");
45              return;
46          }
47          catch (NghiepVuException ex) { _view.ShowError(ex.Message); return; }      // ví dụ: giỏ
            ↪  trống, không đủ quyền
48          catch (Exception ex) { _view.ShowError("Thanh toán thất bại; dữ liệu đã được rollback.\n" +
            ↪  ThongBao(ex)); return; }
49
50          // Bước 2: hóa đơn ĐÃ commit. Lỗi ở bước in/xem trước KHÔNG được báo là thất bại (tránh
            ↪  người dùng bán lại lần hai).
51          _gio.Xoa();
52          try
53          {
54              if (_view.HoiInHoaDon($"Thanh toán thành công!\nMã hóa đơn: HD{hd.MaHD:000000}\nTổng
                ↪  tiền: {hd.Tong:N0} ₫\n\nBạn có muốn xem và in hóa đơn không?"))
55                  _view.ShowInvoicePreview(hd);
56          }
57          catch (Exception ex)
58          {
59              _view.ShowError($"Hóa đơn HD{hd.MaHD:000000} ĐÃ được lưu thành công (không cần thanh
                ↪  toán lại), nhưng không mở được bản xem trước / in hóa đơn.\n\nChi tiết:
                ↪  {ex.Message}\nKiểm tra máy in mặc định rồi in lại từ trang Hóa đơn.");
60          }
61          Start();
62      }
63  }
```

C.4 In hóa đơn: `InvoiceRenderer.cs`

Danh sách mã C.4: InvoiceRenderer.cs – vẽ và phân trang hóa đơn

```
1   using PharmaCare.Models;
2   namespace PharmaCare.Printing;
3
4   /// <summary>MỘT hàm vẽ hóa đơn duy nhất, dùng cho cả Print Preview lẫn in thật (cùng đi qua sự
    ↪  kiện PrintPage). Hóa đơn nhiều dòng tự sang trang.</summary>
5   public static class InvoiceRenderer
6   {
7       /// <summary>Vẽ toàn bộ hóa đơn trên MỘT trang (dành cho hóa đơn ngắn / xem nhanh).</summary>
8       public static void DrawInvoice(Graphics g, Rectangle bounds, HoaDon hd)
9       {
10          int next = 0;
```

```
        DrawInvoice(g, bounds, hd, ref next);
    }

    /// <summary>
    /// Vẽ MỘT trang của hóa đơn. <paramref name="next"/> là chỉ số dòng chi tiết đầu tiên cần vẽ (0 = trang đầu) và được cập nhật sau khi vẽ.
    /// Trả về true nếu còn dòng chưa in (gán vào PrintPageEventArgs.HasMorePages). Tổng cộng chỉ in ở trang cuối.
    /// </summary>
    public static bool DrawInvoice(Graphics g, Rectangle bounds, HoaDon hd, ref int next)
    {
        const float RowH = 22, FooterH = 70;
        g.SmoothingMode = System.Drawing.Drawing2D.SmoothingMode.AntiAlias;
        using var fTitle = new Font("Segoe UI", 16, FontStyle.Bold);
        using var fBold = new Font("Segoe UI", 10, FontStyle.Bold);
        using var fText = new Font("Segoe UI", 10);
        using var fSmall = new Font("Segoe UI", 9, FontStyle.Italic);
        using var pen = new Pen(Color.Black, 1);
        using var teal = new SolidBrush(Ui.Teal);
        using var center = new StringFormat { Alignment = StringAlignment.Center };
        using var right = new StringFormat { Alignment = StringAlignment.Far };
        using var ellipsis = new StringFormat { Trimming = StringTrimming.EllipsisCharacter, FormatFlags = StringFormatFlags.NoWrap };

        bool first = next == 0;
        float y = bounds.Top;
        g.DrawString("+ PHARMACARE", fTitle, teal, new RectangleF(bounds.Left, y, bounds.Width, 32), center); y += 34;
        if (first)
        {
            g.DrawString("HÓA ĐƠN BÁN LẺ", fBold, Brushes.Black, new RectangleF(bounds.Left, y, bounds.Width, 22), center); y += 28;
            g.DrawString($"Số: HD{hd.MaHD:000000}     Ngày: {hd.NgayLap:dd/MM/yyyy HH:mm}", fText, Brushes.Black, bounds.Left, y); y += 22;
            g.DrawString($"Khách hàng: {hd.TenKhach}", fText, Brushes.Black, bounds.Left, y); y += 22;
            g.DrawString($"Người lập: {hd.NguoiLap}", fText, Brushes.Black, bounds.Left, y); y += 28;
        }
        else
        {
            g.DrawString($"HÓA ĐƠN HD{hd.MaHD:000000} (tiếp theo)", fBold, Brushes.Black, new RectangleF(bounds.Left, y, bounds.Width, 22), center); y += 30;
        }

        // cột: Tên thuốc | Lô | SL | Đơn giá | Thành tiền
        float w = bounds.Width;
        float[] x = { bounds.Left, bounds.Left + w * 0.34f, bounds.Left + w * 0.52f, bounds.Left + w * 0.60f, bounds.Left + w * 0.78f, bounds.Right };
        string[] head = { "Tên thuốc", "Lô", "SL", "Đơn giá", "Thành tiền" };
        g.DrawLine(pen, bounds.Left, y, bounds.Right, y); y += 3;
        for (int i = 0; i < 5; i++)
            g.DrawString(head[i], fBold, Brushes.Black, new RectangleF(x[i], y, x[i + 1] - x[i], 22), i >= 2 ? right : ellipsis);
        y += 24; g.DrawLine(pen, bounds.Left, y, bounds.Right, y); y += 4;

        int drawn = 0;
        while (next < hd.ChiTiet.Count)
        {
```

```
            // luôn chừa chỗ cho phần tổng cộng; mỗi trang in ít nhất 1 dòng để không bao giờ lặp vô hạn
            if (drawn > 0 && y + RowH > bounds.Bottom - FooterH) break;
            var c = hd.ChiTiet[next];
            g.DrawString(c.TenThuoc, fText, Brushes.Black, new RectangleF(x[0], y, x[1] - x[0] - 4, RowH), ellipsis);
            g.DrawString(c.SoLo, fText, Brushes.Black, new RectangleF(x[1], y, x[2] - x[1] - 4, RowH), ellipsis);
            g.DrawString(c.SoLuong.ToString(), fText, Brushes.Black, new RectangleF(x[2], y, x[3] - x[2] - 4, RowH), right);
            g.DrawString(c.DonGia.ToString("N0"), fText, Brushes.Black, new RectangleF(x[3], y, x[4] - x[3] - 4, RowH), right);
            g.DrawString(c.ThanhTien.ToString("N0"), fText, Brushes.Black, new RectangleF(x[4], y, x[5] - x[4], RowH), right);
            y += RowH; next++; drawn++;
        }
        y += 4; g.DrawLine(pen, bounds.Left, y, bounds.Right, y); y += 6;

        if (next < hd.ChiTiet.Count)       // còn dòng: sang trang sau, tổng cộng in ở trang cuối
        {
            g.DrawString("... (còn tiếp ở trang sau)", fSmall, Brushes.Gray, new RectangleF(bounds.Left, y, w, 22), right);
            return true;
        }
        g.DrawString("TỔNG CỘNG:", fBold, Brushes.Black, new RectangleF(bounds.Left, y, w * 0.6f, 24));
        g.DrawString($"{hd.Tong:N0} đ", fBold, Brushes.DarkRed, new RectangleF(bounds.Left, y, w, 24), right); y += 40;
        g.DrawString("Cảm ơn quý khách! Thuốc đã bán không đổi trả.", fSmall, Brushes.Black, new RectangleF(bounds.Left, y, w, 22), center);
        return false;
    }
}
```

C.5 Khởi tạo CSDL: `DbSetup.cs`

Danh sách mã C.5: DbSetup.cs – dò máy chủ và tự tạo cơ sở dữ liệu

```
using Microsoft.Data.SqlClient;
using System.Reflection;
using System.Text;
using System.Text.RegularExpressions;
namespace PharmaCare;

/// <summary>
/// Tự dò SQL Server, tự tạo database PharmaCareDB + dữ liệu mẫu + stored procedure (3 script nhúng sẵn trong ứng dụng) nếu chưa có.
/// Nhờ vậy không cần chạy tay file .sql trong SSMS.
/// </summary>
public static class DbSetup
{
    /// <summary>Trả về null nếu thành công, ngược lại trả về nội dung lỗi để hiển thị.</summary>
    public static string? EnsureReady()
    {
```

```
16          var errors = new List<string>();
17          foreach (var server in AppConfig.ServerCandidates())
18          {
19              try
20              {
21                  using (var master = new SqlConnection(AppConfig.Build(server, "master")))
22                  {
23                      master.Open();
24                      switch (CheckState(master))
25                      {
26                          case DbState.CanTaoMoi: RunScript(master, Scripts); break;
                            ↪  // chưa có DB / bản cũ thiếu bảng Users: tạo lại toàn bộ
27                          case DbState.ThieuProcedure: RunScript(master, new[] {
                            ↪  "03_StoredProcedures.sql" }); break; // chỉ thiếu SP: CREATE OR ALTER,
                            ↪  GIỮ NGUYÊN dữ liệu
28                      }
29                  }
30                  var cs = AppConfig.Build(server, AppConfig.Database);
31                  using (var test = new SqlConnection(cs)) { test.Open(); EnsureDemoUsers(test); }
32                  AppConfig.ConnectionString = cs;
33                  AppConfig.ActiveServer = server;
34                  return null;
35              }
36              catch (Exception ex)
37              {
38                  errors.Add($"• {server}\n    {ex.Message.Split('\n')[0].Trim()}");
39              }
40          }
41          var who = $"{Environment.UserDomainName}\\{Environment.UserName}";
42          return "Đã thử các máy chủ sau nhưng đều không thành công:\n\n" + string.Join("\n", errors)
            ↪  +
43                 "\n\nCách xử lý:\n" +
44                 "1) Mở dịch vụ SQL Server (services.msc) và chắc chắn SQL Server (SQLEXPRESS) đang
                   ↪  Running.\n" +
45                 "2) Mở SSMS, xem 'Server name' lúc đăng nhập rồi gán vào AppConfig.Server.\n" +
46                 $"3) Nếu báo thiếu quyền, mở SSMS bằng tài khoản quản trị và chạy:\n   ALTER SERVER
                   ↪  ROLE sysadmin ADD MEMBER [{who}];\n" +
47                 "4) Hoặc dùng SQL login (ví dụ sa): điền AppConfig.SqlUser /
                   ↪  AppConfig.SqlPassword.";
48      }
49
50      /// <summary>Bổ sung tài khoản demo nếu thiếu (ví dụ CSDL được tạo từ bản cũ nên không chạy lại
        ↪  script dữ liệu mẫu).
51      /// Không động đến tài khoản đã có. Lỗi ở bước này không chặn việc mở ứng dụng.</summary>
52      static void EnsureDemoUsers(SqlConnection cn)
53      {
54          const string sql = @"
55  IF NOT EXISTS (SELECT 1 FROM dbo.Users WHERE Username='duocsi')
56      INSERT dbo.Users(Username,PasswordHash,FullName,Role) VALUES('duocsi','PBKDF2$100000$QJoBXA57e⌋
        ↪  naB52Qx2uxtOQ==$C1vkc+VIQUr7CuKJChTmbx/1F29JL0Ynk2MrVsm4KYk=',N'Nguyễn Thị Dược
        ↪  Sĩ','DUOCSI');
57  IF NOT EXISTS (SELECT 1 FROM dbo.Users WHERE Username='quanly')
58      INSERT dbo.Users(Username,PasswordHash,FullName,Role) VALUES('quanly','PBKDF2$100000$eKiDYwMpR⌋
        ↪  JExiEVC9EA6cQ==$7Pvu5tz83/SKnnZyYVaKJBrdkIclHqa8hLNUBfRMwAk=',N'Trần Văn Quản
        ↪  Lý','QUANLYKHO');";
59          try { using var cmd = new SqlCommand(sql, cn); cmd.ExecuteNonQuery(); }
60          catch { /* chỉ là tiện ích cho tài khoản demo */ }
61      }
62
```

```
    enum DbState { CanTaoMoi, ThieuProcedure, SanSang }

    /// <summary>Chưa có database hoặc thiếu bảng Users (bản cũ) => tạo lại; có đủ bảng nhưng thiếu sp_Lo_Nhap => chỉ tạo lại stored procedure (không đụng dữ liệu).</summary>
    static DbState CheckState(SqlConnection cn)
    {
        using var cmd = new SqlCommand(
            "SELECT CASE WHEN DB_ID(@db) IS NULL THEN 0 WHEN OBJECT_ID(@tbl) IS NULL THEN 0 WHEN OBJECT_ID(@sp) IS NULL THEN 1 ELSE 2 END", cn);
        cmd.Parameters.AddWithValue("@db", AppConfig.Database);
        cmd.Parameters.AddWithValue("@tbl", AppConfig.Database + ".dbo.Users");
        cmd.Parameters.AddWithValue("@sp", AppConfig.Database + ".dbo.sp_Lo_Nhap");
        return Convert.ToInt32(cmd.ExecuteScalar()) switch { 0 => DbState.CanTaoMoi, 1 => DbState.ThieuProcedure, _ => DbState.SanSang };
    }

    // Script nhúng sẵn trong ứng dụng, chạy lần lượt: tạo CSDL -> dữ liệu mẫu -> stored procedure
    static readonly string[] Scripts = { "01_CreateDatabase.sql", "02_InsertSampleData.sql", "03_StoredProcedures.sql" };

    static void RunScript(SqlConnection cn, IEnumerable<string> scripts)
    {
        foreach (var name in scripts)
        {
            using var s = Assembly.GetExecutingAssembly().GetManifestResourceStream(name)
                          ?? throw new FileNotFoundException($"Không tìm thấy script {name} nhúng trong ứng dụng.");
            using var r = new StreamReader(s, Encoding.UTF8);
            var sql = r.ReadToEnd();
            foreach (var batch in Regex.Split(sql, @"^[ \t]*GO[ \t]*\r?$", RegexOptions.Multiline | RegexOptions.IgnoreCase))
            {
                if (string.IsNullOrWhiteSpace(batch)) continue;
                using var cmd = new SqlCommand(batch, cn) { CommandTimeout = 60 };
                cmd.ExecuteNonQuery();
            }
        }
    }
}
```

C.6 Script tạo cơ sở dữ liệu

Danh sách mã C.6: 01_CreateDatabase.sql

```
/* 01_CreateDatabase.sql - PharmaCare: tạo database, bảng, khóa chính/ngoại, CHECK, UNIQUE, index.
   Chạy lại được nhiều lần (sẽ xóa dữ liệu cũ). Ứng dụng cũng tự chạy 01 -> 03 khi khởi động lần đầu. */
IF DB_ID(N'PharmaCareDB') IS NULL CREATE DATABASE PharmaCareDB;
GO
USE PharmaCareDB;
GO
IF OBJECT_ID('dbo.vw_ExpiryAlerts','V') IS NOT NULL DROP VIEW dbo.vw_ExpiryAlerts;
IF OBJECT_ID('dbo.sp_GetExpiryAlerts','P') IS NOT NULL DROP PROCEDURE dbo.sp_GetExpiryAlerts;
IF OBJECT_ID('dbo.ReceiptLines','U') IS NOT NULL DROP TABLE dbo.ReceiptLines;
```

```
IF OBJECT_ID('dbo.ReceiptNotes','U') IS NOT NULL DROP TABLE dbo.ReceiptNotes;
IF OBJECT_ID('dbo.InvoiceLines','U') IS NOT NULL DROP TABLE dbo.InvoiceLines;
IF OBJECT_ID('dbo.Invoices','U') IS NOT NULL DROP TABLE dbo.Invoices;
IF OBJECT_ID('dbo.Batches','U') IS NOT NULL DROP TABLE dbo.Batches;
IF OBJECT_ID('dbo.Medicines','U') IS NOT NULL DROP TABLE dbo.Medicines;
IF OBJECT_ID('dbo.Suppliers','U') IS NOT NULL DROP TABLE dbo.Suppliers;
IF OBJECT_ID('dbo.Categories','U') IS NOT NULL DROP TABLE dbo.Categories;
IF OBJECT_ID('dbo.Users','U') IS NOT NULL DROP TABLE dbo.Users;
GO
CREATE TABLE dbo.Users(
 Username VARCHAR(50) NOT NULL CONSTRAINT PK_Users PRIMARY KEY,
 PasswordHash VARCHAR(200) NOT NULL,                       -- PBKDF2-SHA256, không lưu mật khẩu thô
 FullName NVARCHAR(100) NOT NULL,
 Role VARCHAR(20) NOT NULL CONSTRAINT CK_Users_Role CHECK(Role IN ('DUOCSI','QUANLYKHO')),
 IsActive BIT NOT NULL CONSTRAINT DF_Users_Active DEFAULT 1
);
CREATE TABLE dbo.Categories(
 CategoryID INT IDENTITY(1,1) PRIMARY KEY,
 CategoryName NVARCHAR(100) NOT NULL UNIQUE,
 Description NVARCHAR(300) NULL
);
CREATE TABLE dbo.Suppliers(
 SupplierID INT IDENTITY(1,1) PRIMARY KEY,
 SupplierName NVARCHAR(160) NOT NULL CONSTRAINT UQ_Suppliers_Name UNIQUE,
 Phone VARCHAR(20) NULL,
 Email VARCHAR(150) NULL,
 Address NVARCHAR(250) NULL
);
CREATE TABLE dbo.Medicines(
 MedicineID INT IDENTITY(1,1) PRIMARY KEY,
 MedicineCode VARCHAR(30) NOT NULL UNIQUE,
 MedicineName NVARCHAR(160) NOT NULL,
 CategoryID INT NULL REFERENCES dbo.Categories(CategoryID),
 ActiveIngredient NVARCHAR(160) NULL,
 Unit NVARCHAR(30) NOT NULL CONSTRAINT DF_Medicines_Unit DEFAULT N'Viên',
 SellingPrice DECIMAL(18,2) NOT NULL CONSTRAINT CK_Medicines_Price CHECK(SellingPrice>0),
 IsPrescription BIT NOT NULL CONSTRAINT DF_Medicines_Rx DEFAULT 0,
 IsActive BIT NOT NULL CONSTRAINT DF_Medicines_Active DEFAULT 1,
 ImageFileName NVARCHAR(260) NULL,                         -- chỉ lưu TÊN file; ảnh nằm trong thư
 ↪ mục Images cạnh file .exe
 CreatedAt DATETIME2 NOT NULL CONSTRAINT DF_Medicines_Created DEFAULT SYSDATETIME()
);
CREATE TABLE dbo.Batches(
 BatchID INT IDENTITY(1,1) PRIMARY KEY,
 BatchCode VARCHAR(40) NOT NULL UNIQUE,
 MedicineID INT NOT NULL REFERENCES dbo.Medicines(MedicineID),
 SupplierID INT NULL REFERENCES dbo.Suppliers(SupplierID),
 QuantityReceived INT NOT NULL CHECK(QuantityReceived>=0),
 QuantityRemaining INT NOT NULL CHECK(QuantityRemaining>=0),
 CostPrice DECIMAL(18,2) NOT NULL CHECK(CostPrice>=0),
 SellingPrice DECIMAL(18,2) NOT NULL CHECK(SellingPrice>0),
 ReceivedDate DATE NOT NULL,
 ExpiryDate DATE NOT NULL,
 CreatedAt DATETIME2 NOT NULL CONSTRAINT DF_Batches_Created DEFAULT SYSDATETIME(),
 CONSTRAINT CK_Batches_Expiry CHECK(ExpiryDate>=ReceivedDate)
);
CREATE TABLE dbo.Invoices(
 InvoiceID INT IDENTITY(1,1) PRIMARY KEY,
 InvoiceDate DATETIME2 NOT NULL CONSTRAINT DF_Invoices_Date DEFAULT SYSDATETIME(),
```

```
 CustomerName NVARCHAR(160) NOT NULL CONSTRAINT DF_Invoices_Customer DEFAULT N'Khách lẻ',
 TotalAmount DECIMAL(18,2) NOT NULL CHECK(TotalAmount>=0),
 CreatedBy VARCHAR(50) NOT NULL CONSTRAINT FK_Invoices_Users REFERENCES dbo.Users(Username)
);
CREATE TABLE dbo.InvoiceLines(
 InvoiceLineID INT IDENTITY(1,1) PRIMARY KEY,
 InvoiceID INT NOT NULL REFERENCES dbo.Invoices(InvoiceID),
 BatchID INT NOT NULL REFERENCES dbo.Batches(BatchID),
 Quantity INT NOT NULL CHECK(Quantity>0),
 UnitPrice DECIMAL(18,2) NOT NULL CHECK(UnitPrice>=0)
);
CREATE TABLE dbo.ReceiptNotes(            -- phiếu nhập
 ReceiptID INT IDENTITY(1,1) PRIMARY KEY,
 ReceiptDate DATETIME2 NOT NULL CONSTRAINT DF_ReceiptNotes_Date DEFAULT SYSDATETIME(),
 SupplierID INT NULL REFERENCES dbo.Suppliers(SupplierID),
 CreatedBy VARCHAR(50) NOT NULL CONSTRAINT FK_ReceiptNotes_Users REFERENCES dbo.Users(Username),
 TotalAmount DECIMAL(18,2) NOT NULL CHECK(TotalAmount>=0)
);
CREATE TABLE dbo.ReceiptLines(            -- chi tiết phiếu nhập
 ReceiptLineID INT IDENTITY(1,1) PRIMARY KEY,
 ReceiptID INT NOT NULL REFERENCES dbo.ReceiptNotes(ReceiptID),
 BatchID INT NOT NULL REFERENCES dbo.Batches(BatchID),
 Quantity INT NOT NULL CHECK(Quantity>0),
 CostPrice DECIMAL(18,2) NOT NULL CHECK(CostPrice>=0)
);
GO
CREATE INDEX IX_Batches_Medicine_Expiry ON dbo.Batches(MedicineID,ExpiryDate)
↪  INCLUDE(QuantityRemaining,SellingPrice);
CREATE INDEX IX_Invoices_Date ON dbo.Invoices(InvoiceDate);
GO
```

C.7 Thủ tục lưu sẵn

Danh sách mã C.7: 03_StoredProcedures.sql

```
/* 03_StoredProcedures.sql */
USE PharmaCareDB;
GO
-- Lô đã hết hạn / sắp hết hạn trong @DaysAhead ngày / sắp hết hàng (<= @LowStock). Cột đặt tên
↪  tiếng Việt để hiển thị trực tiếp.
CREATE OR ALTER PROCEDURE dbo.sp_Lo_CanhBao @DaysAhead INT = 30, @LowStock INT = 10
AS
BEGIN
 SET NOCOUNT ON;
 DECLARE @t DATE = CAST(GETDATE() AS date);
 SELECT b.BatchCode AS [Mã lô], m.MedicineName AS [Tên thuốc], b.ExpiryDate AS [Hạn dùng],
        DATEDIFF(day,@t,b.ExpiryDate) AS [Còn (ngày)], b.QuantityRemaining AS [Tồn],
        CASE WHEN b.ExpiryDate < @t THEN N'Đã hết hạn'
             WHEN b.ExpiryDate <= DATEADD(day,@DaysAhead,@t) THEN N'Sắp hết hạn'
             ELSE N'Sắp hết hàng' END AS [Trạng thái]
 FROM dbo.Batches b JOIN dbo.Medicines m ON m.MedicineID = b.MedicineID
 WHERE b.QuantityRemaining > 0 AND (b.ExpiryDate <= DATEADD(day,@DaysAhead,@t) OR
 ↪  b.QuantityRemaining <= @LowStock)
 ORDER BY b.ExpiryDate, m.MedicineName;
```

```
END
GO
-- Tồn kho theo từng lô, xếp theo hạn dùng (thứ tự FEFO). @MedicineID = NULL: tất cả thuốc.
CREATE OR ALTER PROCEDURE dbo.sp_Lo_TonTheoLo @MedicineID INT = NULL
AS
BEGIN
 SET NOCOUNT ON;
 DECLARE @t DATE = CAST(GETDATE() AS date);
 SELECT b.BatchCode AS [Mã lô], m.MedicineName AS [Tên thuốc], b.ReceivedDate AS [Ngày nhập],
 ↪  b.ExpiryDate AS [Hạn dùng],
       b.QuantityRemaining AS [Tồn], b.CostPrice AS [Giá nhập], b.SellingPrice AS [Giá bán],
       CASE WHEN b.ExpiryDate < @t THEN N'Đã hết hạn'
            WHEN b.ExpiryDate <= DATEADD(day,30,@t) THEN N'Sắp hết hạn'
            ELSE N'Bình thường' END AS [Trạng thái]
 FROM dbo.Batches b JOIN dbo.Medicines m ON m.MedicineID = b.MedicineID
 WHERE b.QuantityRemaining > 0 AND (@MedicineID IS NULL OR b.MedicineID = @MedicineID)
 ORDER BY m.MedicineName, b.ExpiryDate, b.BatchID;
END
GO
-- Nhập lô (UPSERT): mã lô đã có thì cộng dồn số lượng (giữ nguyên giá bán của lô), chưa có thì
↪  thêm mới. Gọi trong SqlTransaction của ứng dụng.
CREATE OR ALTER PROCEDURE dbo.sp_Lo_Nhap
 @BatchCode VARCHAR(40), @MedicineID INT, @SupplierID INT = NULL, @ReceivedDate DATE, @ExpiryDate
 ↪  DATE,
 @Quantity INT, @CostPrice DECIMAL(18,2), @SellingPrice DECIMAL(18,2), @BatchID INT OUTPUT
AS
BEGIN
 SET NOCOUNT ON;
 IF @Quantity <= 0 THROW 50010, N'Số lượng nhập phải lớn hơn 0.', 1;
 IF @ExpiryDate <= CAST(GETDATE() AS date) THROW 50011, N'Hạn sử dụng phải sau ngày hôm nay.', 1;
 IF @SellingPrice <= 0 THROW 50013, N'Giá bán phải lớn hơn 0.', 1;

 DECLARE @oldMed INT, @oldExp DATE;
 SELECT @BatchID = BatchID, @oldMed = MedicineID, @oldExp = ExpiryDate
 FROM dbo.Batches WITH (UPDLOCK, HOLDLOCK) WHERE BatchCode = @BatchCode;

 IF @BatchID IS NOT NULL
 BEGIN
  IF @oldMed <> @MedicineID THROW 50014, N'Mã lô này đã thuộc về thuốc khác.', 1;
  IF @oldExp <> @ExpiryDate THROW 50012, N'Mã lô đã tồn tại với hạn dùng khác. Hãy kiểm tra lại mã
  ↪  lô hoặc hạn dùng.', 1;
  -- Giá bán của lô GIỮ NGUYÊN (không đổi giá hàng đã có trên kệ); giá nhập = bình quân gia quyền
  ↪  theo số lượng nhập.
  -- Trong UPDATE, vế phải luôn dùng giá trị CŨ của dòng nên QuantityReceived ở công thức giá là số
  ↪  lượng trước khi cộng.
  UPDATE dbo.Batches
     SET CostPrice = CAST((CostPrice * QuantityReceived + @CostPrice * @Quantity) /
     ↪  (QuantityReceived + @Quantity) AS DECIMAL(18,2)),
         QuantityReceived = QuantityReceived + @Quantity, QuantityRemaining = QuantityRemaining +
         ↪  @Quantity,
         SupplierID = ISNULL(@SupplierID, SupplierID)
   WHERE BatchID = @BatchID;
 END
 ELSE
 BEGIN
  INSERT dbo.Batches(BatchCode,MedicineID,SupplierID,QuantityReceived,QuantityRemaining,CostPrice,
  ↪  SellingPrice,ReceivedDate,ExpiryDate)
  VALUES(@BatchCode,@MedicineID,@SupplierID,@Quantity,@Quantity,@CostPrice,@SellingPrice,@Received
  ↪  Date,@ExpiryDate);
```

```
67    SET @BatchID = SCOPE_IDENTITY();
68   END
69  END
70  GO
```

Tài liệu tham khảo

[1] Microsoft. .net documentation. https://learn.microsoft.com/dotnet/.

[2] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[3] Microsoft. Windows forms documentation. https://learn.microsoft.com/dotnet/desktop/winforms/.

[4] Microsoft. Sql server technical documentation. https://learn.microsoft.com/sql/sql-server/.

[5] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, 7th edition, 2016.

[6] Microsoft. Ado.net overview. https://learn.microsoft.com/dotnet/framework/data/adonet/.

[7] Mike Potel. MVP: Model-View-Presenter. The Taligent Programming Model for C++ and Java. Technical report, Taligent Inc., 1996.

[8] Martin Fowler. GUI Architectures. https://martinfowler.com/eaaDev/uiArchs.html.

[9] K. Moriarty, B. Kaliski, and A. Rusch. PKCS #5: Password-Based Cryptography Specification Version 2.1. Technical Report RFC 8018, IETF, 2017.

[10] OWASP Foundation. Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.